

A Release Test Is Not Runtime Authority: Evidence Reuse at Runtime-Assurance Authority Transitions

Abstract

Learned controllers are commonly commissioned behind runtime assurance: a trusted shield or safety filter holds action authority while the policy adapts, so every training transition satisfies the safety certificate. Deployments then retire that authority and release the raw policy after a finite check. We show that the check does not transfer across this retirement. Evidence collected under one authority is reused to authorize a different subject over a longer horizon, a flaw we call evidence reuse across an authority transition.

The gap does not require an adversary. In a controlled Safe-Control-Gym study, snapshots that passed a five-step release check failed on 2 of 120 snapshot–state pairs once released raw, whereas the same prediction retained at runtime failed on none; a one-step filter did not suffice. An upstream attacker amplifies the gap rather than creating it: an adversary holding only bounded write access to reward records raised raw-release failures to 27 of 120 with no training-time violation, and raised raw-policy violations at retirement by factors of 3.6 and 1.6 on a published third-party shield-retirement lifecycle. Retained predictive authority switched before all 27 failures. Trusted recomputation closes this channel; it does not close the contract gap the channel exploits.

1 Introduction

Runtime assurance is the standard way to let a controller learn without letting it crash the plant. A Simplex decision module, a runtime shield, or a model-predictive safety filter holds action authority during adaptation: it replaces unsafe policy proposals with actions from a trusted baseline, so every transition collected while the policy trains satisfies the safety certificate [1–3]. Deployments do not keep that machinery forever. Published lifecycles and ordinary engineering practice retire the protective authority once commissioning succeeds and promote the raw policy snapshot to a lower-latency control target, which removes permanent computation, intervention, and dependence on a backup service [4, 5]. A finite release

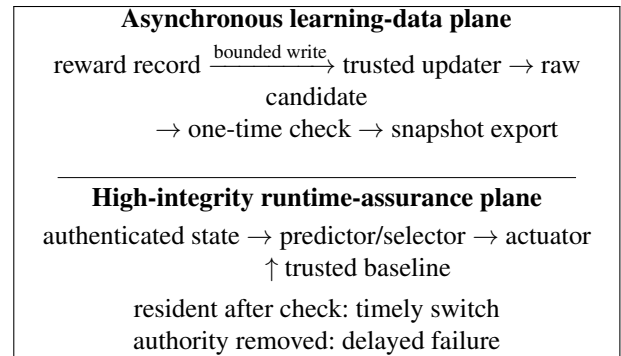


Figure 1: Evaluated trust boundary. The benchmark begins after bounded reward-record access is obtained; it does not model the initial service compromise. Upstream provenance or recomputation removes this attack path, whereas resident predictive authority contains its downstream physical consequence.

check — a bounded-horizon prediction, a certified rollout, a commissioning test — authorizes that promotion.

We show that the release check does not transfer across the retirement it authorizes. Protection makes the executed composition safe; it does not make the raw policy safe. The same finite check is sound while it retains authority and unsound once it is spent at release, because release changes the subject being authorized, the authority enforcing the decision, and the horizon over which the evidence must hold. We call the underlying flaw *evidence reuse across an authority transition*: evidence collected during protected adaptation is reused to authorize a different subject, under a different authority, with different provenance, information, and horizon.

The gap is measurable without an adversary. In our controlled study, snapshots that passed a five-step release check failed on 2 of 120 snapshot–state pairs once released raw, whereas the identical prediction retained at runtime failed on none. Residence alone is not the remedy: on a disjoint cohort, a one-step resident kernel failed on 14 of 71 accepted pairs and reached an empty kernel 41 times. What separates these outcomes is whether a check with enough lookahead to leave switching lead time keeps authority at runtime.

An upstream adversary widens the gap rather than creating it. While the shield masks unsafe proposals, corrupted update data steers the learner without producing a single physical violation, and the damage stays latent until authority is removed: bounded reward-record edits raised the same 120-pair count from 2 to 27. We call this *shield-retirement poisoning*. Its applicability is narrow by construction (Section 3): where the runtime plane can recompute reward from authenticated transitions, trusted recomputation removes the channel outright, as our frozen integrity audit confirms. The contract gap survives that defense.

The attacker we evaluate is deliberately weak. Operational technology (OT) guidance separates runtime-control assets from the data-management services and historians that exchange information across zones [6, 7], and our threat model begins after compromise of a reward-ingestion principal in that learning-data plane. This is an explicit split-trust precondition: the attacker may modify bounded scalar reward records consumed by a policy-gradient updater, and nothing else. Authenticated runtime states, actions, learner code, parameters, optimizer state, certificates, and deployment states remain outside its write authority (Fig. 1).

Existing mechanisms address neighboring problems. Simplex and its neural and barrier-based successors keep a decision module resident [1, 8, 9]. Policy repair and safe projection constrain a learned policy before deployment [10–12]. Reward-poisoning work studies how adversarial learning signals redirect updates, but not behind a shield. None treats the release decision itself as the object of study: the unresolved composition is a transition in which evidence produced under retained authority is spent once, at release, to authorize a raw policy that will run without it.

1.1 Evidence and Contributions

We separate the contract gap from the channel that widens it. The controlled Safe-Control-Gym study supplies the counts above under a five-step check that accepted every candidate, and adds a locked horizon sweep placing the usable lookahead between an insufficient one-step check and an over-conservative twenty-step one. The channel is not confined to our harness: Carr et al.’s published workflow trains a policy behind a Storm belief-support shield and later retires the shield’s authority [4], and adding bounded reward-record poisoning only during the protected bootstrap increased raw-policy violations at retirement by factors of 3.6 and 1.6 in two of three provenance-complete paired REINFORCE training runs.

Our contributions are:

- We formalize release-time acceptance as an evidence tuple over subject, authority, provenance, information, and horizon, and show that a deployment must re-establish every component the authority transition changes.
- We measure the resulting gap without an adversary, separating one-time raw release from resident predictive authority, one-step filtering, and commit admission, and locating the lookahead at which a retained check is neither insufficient nor over-conservative.
- We instantiate one upstream channel that widens the gap — bounded reward-record writes — in the controlled study and in a third-party shield-retirement lifecycle, and report the integrity controls that close that channel while leaving the gap open.
- We characterize the attack’s empirical boundary across systems, learners, integrity controls, and deployment contracts: high retirement-state disagreement excludes every observed positive outcome in the locked corpus, while low disagreement defines an opportunity region without guaranteeing success.

2 Background and Problem Model

2.1 Attacked Histories and Plausible States

We consider a discrete-time cyber-physical system

$$x_{t+1} = f(x_t, u_t, w_t), \quad \tilde{y}_t = h(x_t) + a_t, \quad (1)$$

where x_t is the physical state, u_t is the control input, $w_t \in \mathcal{W}$ is ordinary process uncertainty, and a_t is a false-data injection (FDI) signal on the measurement channel. The controller observes the attacked history

$$h_t = (\tilde{y}_{0:t}, u_{0:t-1}, \tilde{\ell}_{0:t}), \quad (2)$$

where $\tilde{\ell}_t = \ell_t + b_t$ is a learning signal modified by update-data corruption b_t . The general model covers rewards, logs, and auxiliary learning signals; the end-to-end experiments specialize this channel to bounded scalar reward-record edits. Sensor FDI creates uncertainty about the physical state, whereas update-data corruption can redirect learning even with trusted state observations. Either channel can change the action map produced from h_t .

Following severe sensor-attack safety work [13], an attacked history may correspond to multiple physical states. For an attack family $\mathcal{A}$, define the plausible-state set

$$X_{\mathcal{A}}(h_t) = \{x : \exists E \in \mathcal{A}, H_t(E) = h_t, x_t(E) = x\}. \quad (3)$$

This set defines the state semantics of a certificate under strategic FDI: a certificate evaluated only at a nominal estimate $\hat{x}(h_t)$ does not cover another compatible state in $X_{\mathcal{A}}(h_t)$.

2.2 Online-Updated Controllers and Certified Kernels

The controller is a history-dependent policy π_{θ_t} that selects $u_t = \pi_{\theta_t}(h_t)$ and updates according to $\theta_{t+1} = U(\theta_t, h_t)$. An update is *nontrivial* when it changes the controller’s action

map on a reachable or admissible history; parameter changes that preserve the action map are behaviorally trivial.

Let Γ denote an invariant-set, barrier, reachability, or runtime-shield certificate. At history h , the certificate induces actions that are valid for every plausible state:

$$K_\Gamma(h) = \{u \in \mathcal{U} : u \text{ satisfies } \Gamma \text{ for all } x \in X_{\mathcal{A}}(h)\}. \quad (4)$$

We call $K_\Gamma(h)$ the *certified kernel*. Severe-attack safety filters construct or act within such kernels for fixed controllers. On-line learning additionally requires admission of the updated action map produced by U .

Definition 1 (Certified online learning). *A mechanism is a certified online learner at history h if it accepts a certificate for $\pi_{\theta_{t+1}}$, where $\theta_{t+1} = U(\theta_t, h)$. The accepted certificate asserts that the next certified action is sound for every state in $X_{\mathcal{A}}(h)$.*

Definition 2 (Learning-freeze frontier). *For a family of histories parameterized by attack strength or ambiguity radius, the learning-freeze frontier is the boundary at which $K_\Gamma(h)$ becomes empty, the candidate updated action leaves $K_\Gamma(h)$, or the candidate parameter leaves the corresponding certified set $\Theta_\Gamma(h)$. Beyond this boundary, a sound certified learner must reject the certificate, freeze the update, or obtain extra trusted state information.*

2.3 Assurance-Contract Failure Modes

Four failure modes arise at different points in the assurance lifecycle. Under *nominal-state certification*, a certificate evaluates an estimate $\hat{x}(h)$ or a reported sensor value, so under FDI an accepted action may be safe for $\hat{x}(h)$ but unsafe for another state in $X_{\mathcal{A}}(h)$; severe-attack safety filters address this state-semantics failure. Under *action-only filtering*, a robust filter projects the current action into $K_\Gamma(h)$ when the kernel is nonempty, which protects the applied action at that step but leaves the learner update a separate object: if $\pi_{\theta_{t+1}}(h)$ leaves $K_\Gamma(h)$, the certificate covers the filtered action but not the updated action map. Under *uncertified learning after rejection*, an empty kernel lets a severe-attack filter reject the action or enter fail-safe mode while an update computed from the same attacked history falls outside the accepted certificate lifecycle.

The fourth mode is the subject of this paper. Under *protected adaptation with unsafe deployment*, an adaptation shield keeps every data-collection transition safe while corrupted reward records influence the learner. The trusted update rule then produces an unsafe raw policy without any physical failure during adaptation. Releasing that policy after shield retirement exposes the delayed effect, so raw release requires evidence for the policy that receives action authority.

These modes require separate metrics: accepted certificates whose applied action is unsafe for some plausible state, accepted certificates for which the updated raw policy leaves the kernel, and nontrivial updates after certificate rejection.

End-to-end studies add delayed deployment violations, admission coverage, paired clean/poison outcomes, update fraction, intervention, reward, and certificate runtime.

3 Threat Model

3.1 System Roles and Deployment Contracts

The defender deploys an online-updated controller π_{θ_t} and a certificate mechanism; a severe-attack-aware estimator or filter returns a plausible-state set $X_{\mathcal{A}}(h_t)$ and certified action kernel $K_\Gamma(h_t)$. A trusted state anchor, backup controller, or external fail-safe may also be available, but each is an explicit architectural assumption and cost.

The evaluation instantiates four contracts. *Permanent raw release* removes runtime authority after a one-time five-step check: protected commissioning is followed by snapshot export and promotion to a lower-latency control target, which avoids permanent computation, intervention, and dependence on a backup service. *Commit admission* certifies a candidate snapshot on a declared state set and horizon before raw release. *Resident predictive authority* repeats the five-step check online and can transfer control to the trusted baseline. The *one-step resident kernel* applies a current-state action filter and serves as a negative control. A sound filter that remains online instead certifies the composition $F \circ \pi_{\theta+}$. Each name denotes a distinct evidence scope and authority contract, and Section 6 measures their tradeoffs.

3.2 Split-Trust Learning Pipeline

The least-privilege model separates a high-integrity runtime-assurance plane from an asynchronous learning-data plane. The runtime plane owns the authenticated plant state, action selector, actuator interface, trusted baseline, and certificate outcome. The learning plane consumes recorded transitions and scalar rewards through telemetry, a historian, a replay buffer, or an external key-performance-indicator interface. OT guidance recognizes this separation and the exchange of historian data across zones [6, 7]. The benchmark models the resulting interface without assuming a particular product.

Table 1 states the evaluated write boundary. The attack begins after compromise of a principal with bounded reward-field write access, which permits post-computation, pre-update mutation by a replay writer or broker. A compromised reward producer is equivalent when it has the same bounded output authority; in that case source authentication establishes origin but not reward semantics.

Applicability condition. The learning-data plane is deliberately reachable: historians, ingestion brokers, replay buffers, and KPI services exist so that offline consumers can read and enrich operational records, and OT guidance treats cross-zone historian exchange as an expected data flow [6, 7]. A principal on that path holds bounded write authority over a

Table 1: Evaluated write boundary. “Trusted” means outside attacker write authority, not formally verified.

Object	Defender assumption	Attacker access
Runtime state, action, plant	Authenticated control path	No write
Baseline, selector, certificate	Trusted runtime code/state	No write
Transitions and learner code	Trusted after collection	Read only
Reward record at ingestion	No origin/semantic check	Bounded scalar write
Parameters and optimizer state	Written only by updater	No write
Held-out states and release rule	Fixed, honest evaluation	No write or selection

scalar field long before it holds any authority over the control path, which is why we grant the attacker nothing else. The channel is only interesting when the runtime plane cannot recompute the reward from authenticated transitions. That is the case when reward depends on delayed quality labels, operator feedback, manual disposition codes, or an external KPI the runtime plane does not observe. It is not the case for rewards that are deterministic functions of authenticated state and action: there, trusted recomputation removes the channel outright, as our frozen integrity audit confirms (Section 6). We therefore state applicability as a requirement rather than an assumption. This attack surface exists exactly where reward semantics cannot be reconstructed inside the high-integrity plane.

3.3 Attack Channels and Capabilities

The general history model permits two attack channels. Observation FDI a_t changes the state semantics of the history and enlarges $X_{\mathcal{A}}(h_t)$. Update-data corruption b_t modifies learning signals and can redirect $U(\theta_t, h_t)$ with trusted state observations. Either channel can alter the next action map.

The end-to-end learner experiments use the narrower reward-record channel. The white-box, batch-aware attacker observes the current learner and adaptation batch, but not the held-out deployment-state choice. It modifies only scalar rewards before an ordinary policy-gradient update, subject to a declared per-step L_∞ budget. It cannot write actuator commands, physical states, observations, dynamics, gradients, policy parameters, optimizer state, deployment states, or certificate outcomes. The adaptation selector observes each proposed action before the plant step and retains physical authority during data collection.

3.4 Evidence Transfer Across Authority Transitions

The contracts above differ in their evidence obligations. We represent the evidence used by an assurance decision as

$$E = (s, a, p, i, \tau), \quad (5)$$

The *subject* s is the raw policy π_θ or the assured composition $F \circ \pi_\theta$. The *authority* a identifies the resident module that constrains applied actions, or its absence. The *provenance* p records whether learning signals came from a trusted or adversarial update process. The *information* i describes the state, observation, belief, or filtered action available at decision time. The *horizon* τ specifies the interval over which the evidence bounds behavior.

A contract transition $C_{\text{train}} \rightarrow C_{\text{deploy}}$ may change one or more components, and evidence transfers only after every changed component is re-established under C_{deploy} . Evidence about the composition does not bound the raw policy, and a raw-policy audit does not certify the composition (*subject*). Evidence collected while a resident module masked unsafe proposals describes the masked process, not the raw policy after the module is removed (*authority*). A policy trained on an adversarial update log is not the output of the trusted process the evidence was computed on (*provenance*). Behavior under the authority’s information set does not determine behavior under the deployment observation (*information*). A finite evaluation pass does not bound an unbounded future (*horizon*).

Proposition 1 (Transfer obligation). *Let E be evidence collected under contract C_{train} , and let the transition $C_{\text{train}} \rightarrow C_{\text{deploy}}$ change a nonempty subset of the components of the evidence tuple. Acceptance under C_{deploy} is unsound unless each changed component’s evidence obligation is re-established under C_{deploy} .*

The proposition supplies the security model used throughout the paper. Independent transfer evidence re-establishes obligations before authority removal [5]. Resident Simplex mechanisms preserve the authority component [1, 8, 9]. Carr et al.’s lifecycle supplies a published transition from shielded bootstrap to raw-policy authority [4]. Our case study adds adversarial update provenance to that transition, and the controlled study isolates the resulting mechanism.

3.5 Integrity Assumptions and Escape Conditions

The vulnerable configuration supplies neither origin integrity for the reward record nor trusted reward recomputation. Binding a reward, transition, action, producer identity, and time to an authenticated append-only record blocks post-ingestion mutation, and recomputation by a trusted principal completely

blocks the evaluated channel whenever reward is a deterministic function of authenticated transitions. Under the applicability condition above, the system instead needs a trusted producer, redundant validation, or a detector; a signature applied to an already malicious value is insufficient.

These controls act at different boundaries: provenance and recomputation remove the upstream precondition, a task-aware detector can reject suspicious batches under its distributional and semantic assumptions, commit admission restricts which learned snapshot can be released, and resident predictive authority contains the physical consequence after deployment. In the frozen audit, trusted recomputation achieved true-/false-positive rates (TPR/FPR) of 1.000/0 and simple task-aware checks achieved TPR above 0.90 and FPR at most 0.036, making the effective attack readily detectable under those assumptions.

3.6 Security and Availability Goals

We separate four attacker or failure objectives. An *unsafe certified action* occurs when the mechanism accepts an applied input that is unsafe for some state in $X_{\mathcal{A}}(h_t)$. A *stale certified policy* occurs when the current applied input is filtered but the accepted raw updated action map leaves the kernel. *Uncertified learning* occurs when a nontrivial update continues after certificate rejection. *Forced freeze* occurs when the attacker expands the plausible set or redirects the update until sound learning must stop or re-anchor. The defender also measures availability through admitted state coverage, accepted update fraction, task reward, filter intervention, rejection, certificate latency, and trusted-anchor use; useful learning requires paired task improvement over always-freeze in addition to physical safety.

Severe-attack filters, secure estimators, and fault-tolerant control-barrier function (CBF) methods can preserve safety when their assumptions keep $K_{\Gamma}(h_t)$ nonempty [13–15]. Out of scope are service exploitation, reward-monitor evasion, and compromise of trusted recomputation; Section 8.1 states the remaining validity boundaries.

4 Contract-Aware Update Analysis

This section develops the update conditions used by the empirical studies. The current- and future-history gates distinguish action filtering from certified learning, and the same conditions state what a deployment contract must re-establish when it stops applying them. A reward-influence summary then records how a bounded reward edit can move a linear release-gate margin within one policy-gradient batch; the evaluated multi-batch attack remains the fixed target-shaping rule described in Section 6. All proofs are deferred to Appendix A.

4.1 Current- and Future-History Gates

The certificate condition has two levels. The current-history condition requires the updated action at the observed history to remain certified. The lifecycle condition extends this requirement to future attacked histories reachable after the update.

Lemma 1 (Current-history gate). *Let $M = (\pi_{\theta}, U, \text{Cert})$ be a history-dependent online controller. For an attacked history h , let $X_{\mathcal{A}}(h)$ be the plausible-state set and $K_{\Gamma}(h)$ its certified kernel. If M accepts a sound certificate for $\pi_{\theta_{t+1}}$, where $\theta_{t+1} = U(\theta_t, h)$, then*

$$\pi_{\theta_{t+1}}(h) \in K_{\Gamma}(h). \quad (6)$$

Consequently, if $K_{\Gamma}(h) = \emptyset$, then M cannot both accept the certificate and allow a nontrivial online update at h .

An action filter that returns a certified input at h does not certify the update that produced it. The filtered actuator channel can satisfy the certificate while $\pi_{\theta^+}(h) \notin K_{\Gamma}(h)$, so the updated learner remains uncertified; Appendix A states this as Proposition 3.

These gates are local, but a policy update also determines actions at future histories. Let $\mathcal{H}_{\mathcal{A}}(h, \theta^+)$ denote the attacked histories reachable after h under π_{θ^+} and attack family $\mathcal{A}$, and define the certified parameter set $\Theta_{\Gamma}(h) = \{\theta : \forall h' \in \mathcal{H}_{\mathcal{A}}(h, \theta), \pi_{\theta}(h') \in K_{\Gamma}(h')\}$.

Theorem 1 (Future-history lifecycle gate). *Let $\theta^+ = U(\theta_t, h)$ be a candidate online update. Suppose M accepts a sound lifecycle certificate for π_{θ^+} from history h over attack family $\mathcal{A}$. Then*

$$\theta^+ \in \Theta_{\Gamma}(h), \quad (7)$$

equivalently,

$$\forall h' \in \mathcal{H}_{\mathcal{A}}(h, \theta^+), \quad \pi_{\theta^+}(h') \in K_{\Gamma}(h'). \quad (8)$$

If membership fails for a reachable h' , accepting the update creates a stale certified policy; if $K_{\Gamma}(h') = \emptyset$, certified learning must reject, freeze, roll back, or obtain trusted information that changes $X_{\mathcal{A}}(h')$.

Proposition 2 (Linear-policy parameter gate). *Consider a finite abstraction of future attacked histories indexed by $z \in \mathcal{Z}$ and a linear policy $\pi_{\theta}(z) = \theta^{\top} \phi(z)$. Suppose each certificate produces an interval kernel $K_{\Gamma}(z) = [\ell_z, u_z]$. The lifecycle condition over $\mathcal{Z}$ is then equivalent to*

$$\theta^{\top} \phi(z) \leq u_z, \quad -\theta^{\top} \phi(z) \leq -\ell_z, \quad \forall z \in \mathcal{Z}. \quad (9)$$

Optional box constraints on θ preserve a convex polytope. Rejecting candidates outside this set, or projecting onto it, is sound for the finite abstraction.

Proposition 2 is exact only on its listed grid. Appendix B states the coverage, regularity, model-error, and margin assumptions under which it lifts to continuous reachable histories, together with a plausible-set monotonicity result, the

kernel instantiations for invariant and barrier certificates, and a minimal counterexample separating fixed filtering from certified learning.

4.2 Reward Influence on a Release Gate

The upstream channel of Section 3 reaches these halfspaces through a single policy-gradient batch: editing a reward after collection leaves the states, actions, and score rows of that batch unchanged, so a bounded edit produces gate-directed parameter motion within the batch. Appendix C makes this exact, reducing the positive support of an admissible reward edit against a linear release halfspace to a monotone family of second-order-cone feasibility problems, valid under global Euclidean gradient-norm clipping; if that support does not exceed the current margin, no admissible edit can cross the halfspace in the batch. The result is confined to one batch and does not explain the multi-batch attack we evaluate, which is the fixed target-shaping rule of Section 6. Coordinatewise parameter clipping remains outside the exact support computation.

4.3 Certificate-Gated Update and Deployment Contracts

The resulting update rule first obtains $X_{\mathcal{A}}(h)$ from a severe-attack-aware estimator or safety filter and computes $K_{\Gamma}(h)$. An empty kernel rejects the certificate and freezes the update. Otherwise the mechanism accepts, projects, constrains, or rejects the candidate according to the updated action’s kernel membership. For linear policies the admission test is the convex halfspace system of Proposition 2; a grid certifies only its listed points unless the cover, regularity, and model-error premises of Appendix B justify a robust margin. We call this enforcement layer LifecycleGate.

The layer realizes the deployment contracts of Section 3. They differ in *when* the admission test is applied, not in what it tests: a one-time release contract evaluates Theorem 1 once, at h , and then executes π_{0+} untested, whereas a resident contract re-evaluates it at every subsequent history. The two agree exactly when the finite abstraction used at h covers every history the released policy later reaches — the premise Appendix B isolates, and one that release-time evidence cannot establish on its own. Commit admission certifies a candidate on a declared state set and horizon, backtracking along the connected segment from the last admitted snapshot to exclude disconnected safe islands, at the cost of coverage and model-mismatch risk; resident mechanisms instead pay intervention, rejection, and online cost. An empty plausible-state kernel, an uncertified snapshot, model failure, or solver failure triggers a fail-closed freeze or rollback, and a trusted anchor can resume learning by changing the justified plausible-state set at an explicit infrastructure cost. Because membership

is enforced independently of the task objective, a safe update may be neutral or harmful: accepted parameter distance measures update availability, whereas paired task outcomes against always-freeze measure learning utility under benign change.

5 Third-Party Shield-Retirement Case Study

This section examines the same integrity boundary in an independently published lifecycle that retires runtime authority. Carr et al. [4] (AAAI 2023) train a policy-gradient agent behind a Storm belief-support shield in a partially observable gridworld, then remove the shield suddenly or smoothly and compare the released policy with retained-shield and no-shield controls. We use the official public implementation and modify only learner-visible reward records: the attacker writes $r' = r + \xi$, with $|\xi| \leq \delta$ per record, before the trusted update. States, observations, shield semantics, dynamics, learner code, hyperparameters, and evaluation follow upstream, and the artifact itemizes every instrumentation and attack patch. REINFORCE training runs for 5000 episodes with retirement at episode 1000; the TF-Agents proximal policy optimization (PPO) pipeline retires the shield after 10^5 environment steps.

Lifecycle fidelity. On the obstacle domain ($N=6$), the clean condition reproduces the qualitative ordering of Carr et al.’s Table 1 for REINFORCE: shield retained 0/0 violations during/after training, smooth 326/675, sudden 3277/3784, and no-shield 3798/3784. Shield < smooth $\ll$ sudden $\leq$ no-shield is the ordering reported by Carr et al., establishing the lifecycle fidelity of the instrumented implementation.

Causal isolation at the retirement boundary. Under a locked protocol, poisoning is confined to the shielded bootstrap: reward-record mutation is active only while the shield enforces the action mask, stops at retirement, and all subsequent learning is clean. We evaluate the raw policy at the switch instant before any unshielded learning (1000 episodes) and again after the full budget (5000 episodes). Each independently trained clean/poisoned pair is one training-level replicate; configured seeds serve only as run indices. A pair is *positive* when the poisoned at-retirement rate exceeds its clean pair by at least 0.15 and the paired McNemar exact test gives $p < 0.01$; all other outcomes are *non-positive*, and *protective* denotes a lower poisoned rate.

Two of the three provenance-complete pairs had higher raw-policy violation rates at retirement under poisoning, by factors of 3.6 and 1.6. The fresh third pair was unchanged ($0.241 \rightarrow 0.242$). Physical violations remained zero throughout protected execution. The positive outcomes located latent policy damage before unshielded learning. The fresh pair showed that the fixed attack can fail in a low-disagreement retirement state. After 4000 episodes of clean post-retirement

Table 2: Retirement-boundary causal isolation on the third-party lifecycle (obstacle, REINFORCE, sudden). Each row is a provenance-complete paired training run. Poisoning stops at retirement, and all later training is clean. At-retirement evaluates the raw policy at episode 1000; final evaluates it at episode 5000. Rates are violation fractions.

configured seed	at retirement		final	
	clean	poisoned	clean	poisoned
1	0.216	0.777	0.236	0.235
2	0.484	0.763	0.234	0.751
3 ^a	0.241	0.242	0.747	0.228

^aAn earlier run-index-3 clean summary duplicated a separate fix-check record and is excluded. This row reports a fresh clean/poisoned pair generated under the locked configuration. The archived poisoned realization (0.748 at retirement) lacks a provenance-valid clean endpoint and is excluded from paired inference.

learning, one positive pair recovered (0.235 vs. clean 0.236), one remained worse (0.751 vs. 0.234), and the fresh non-positive pair reversed (0.228 vs. 0.747). We denote clean raw-policy/shield disagreement at retirement by D_{ret} . In the two positive pairs, D_{ret} was 0.025 and 0.047 and rose to 0.113 and 0.109 under poisoning. The fresh non-positive pair also had low clean disagreement ($D_{\text{ret}} = 0.013$), separating available headroom from attack success.

Learner and budget boundary. The same contrast attack was also evaluated on the third-party PPO and off-policy soft actor-critic (SAC) pipelines, across attack budgets and attack shapes, and on a second gridworld of the same suite. It raised post-removal violations in individual PPO and SAC training realizations without producing reliable success within either family, its dose response depended on the realized retirement state rather than the nominal seed, and the second environment reproduced the lifecycle ordering while remaining realization-dependent under attack. Appendix F reports these batteries in full.

Escape condition. Under the same full-scope poisoning used in the full-phase battery, a retained shield kept physical violations at zero before and after training in all three paired runs. The corresponding sudden-retirement runs exposed final violation fractions of 0.490, 0.752, and 0.744 after removal. Resident authority therefore contained the downstream consequence of full-scope contamination; removing it exposes the same consequence as an unsafe released policy.

Interpretation. Carr et al.’s workflow supplies the lifecycle component: protected bootstrap followed by retirement of the shield’s action authority, evaluated under unpoisoned learning. The controlled study in Section 6 supplies the adversarial-provenance mechanism. Their composition exposes an integrity boundary at the authority transition. Absolute rates

differ across instrumented and non-instrumented code versions because the unmasked-disagreement probe requires an independent belief tracker; all clean-vs-poisoned comparisons are paired within a locked code version and are unaffected by that change.

Training outcomes varied across platforms. A ten-run REINFORCE shield-on-only battery on Linux produced three positive low-clean-violation realizations ($0.240 \rightarrow 0.533$, $0.234 \rightarrow 0.744$, $0.229 \rightarrow 0.763$) and seven ceiling realizations with no added damage, whereas the fresh pair in Table 2 was non-positive despite low clean disagreement. The same configured seed and code can enter different trajectory modes across hosts or backends, so we treat each independently trained clean/poisoned pair as a replicate and use nominal seeds only as run indices. Episode-level McNemar tests describe outcomes conditional on a trained pair; the multi-run batteries provide the training-level replication evidence.

Across 53 locked paired runs spanning both scopes, all three learner families, and both domains, 12 of 20 runs with clean at-retirement violation below 0.5 were positive, whereas none of 33 ceiling runs was positive (Appendix F, Fig. 7). Low clean violation therefore marks available headroom but does not determine the outcome.

The 50 obstacle runs with disagreement instrumentation give a sharper retirement-state marker. Clean D_{ret} is strongly associated with clean violation (Spearman 0.976, $p = 1.3 \times 10^{-33}$) and negatively associated with attack headroom (Spearman -0.547 , $p = 3.9 \times 10^{-5}$). Every positive outcome had clean $D_{\text{ret}} \leq 0.047$, while non-positive outcomes extended to 0.111. Under the fixed bands, all 11 positive outcomes fell in the low-disagreement region ($D_{\text{ret}} < 0.06$), and none of 32 high-disagreement runs was positive; the area under the receiver operating characteristic curve (AUC) was 0.953 (Mann–Whitney $p = 5.5 \times 10^{-6}$). High disagreement thus acted as a strong exclusion condition in this corpus, whereas low disagreement defined an opportunity region: eleven of 18 low-band corpus runs were positive, and the fresh pair, analyzed separately from the 50 disjoint namespaces, gave an additional low-band non-positive realization. Appendix F gives the family-stratified counts and the band chronology. Because D_{ret} is measured after the shielded bootstrap, it characterizes a realized retirement state rather than an attacker-side trigger, and the AUC describes discrimination within the locked corpus rather than external classifier validation.

6 Controlled Mechanism Study

The case study located the authority-transition vulnerability in an independently published lifecycle; this section isolates its mechanism in Safe-Control-Gym [16]. A prospective contract-separation audit first tests whether finite release evidence supports both permanent raw release and resident predictive authority, with and without the attacker, using a one-

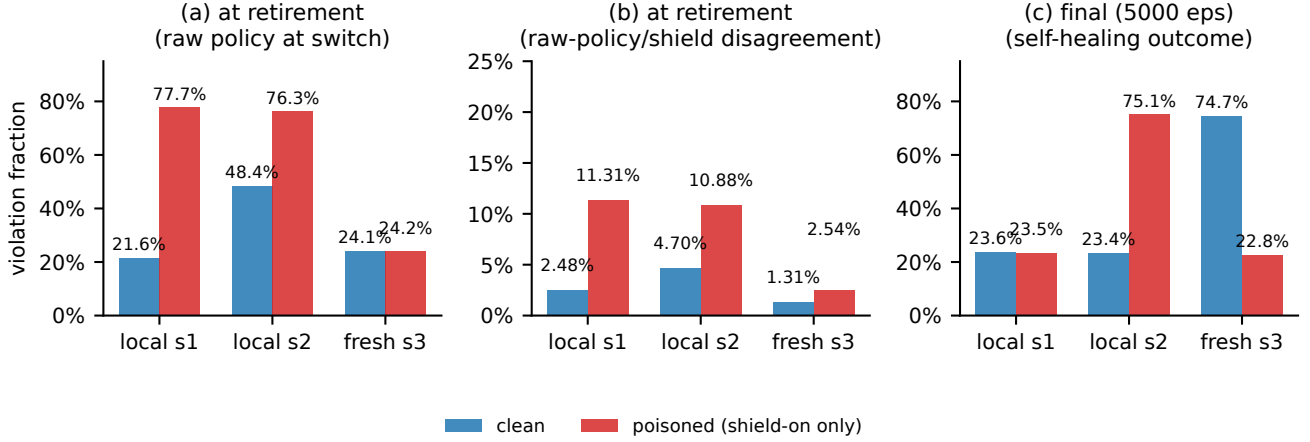


Figure 2: Causal isolation on the third-party lifecycle (REINFORCE, obstacle, sudden, provenance-complete paired runs). (a) At-retirement violation fraction of the raw policy; (b) at-retirement raw-policy/shield disagreement; (c) final violation fraction after clean post-retirement learning. Poisoning is confined to the shielded bootstrap phase. The run-index-3 bars show the fresh pair in Table 2.

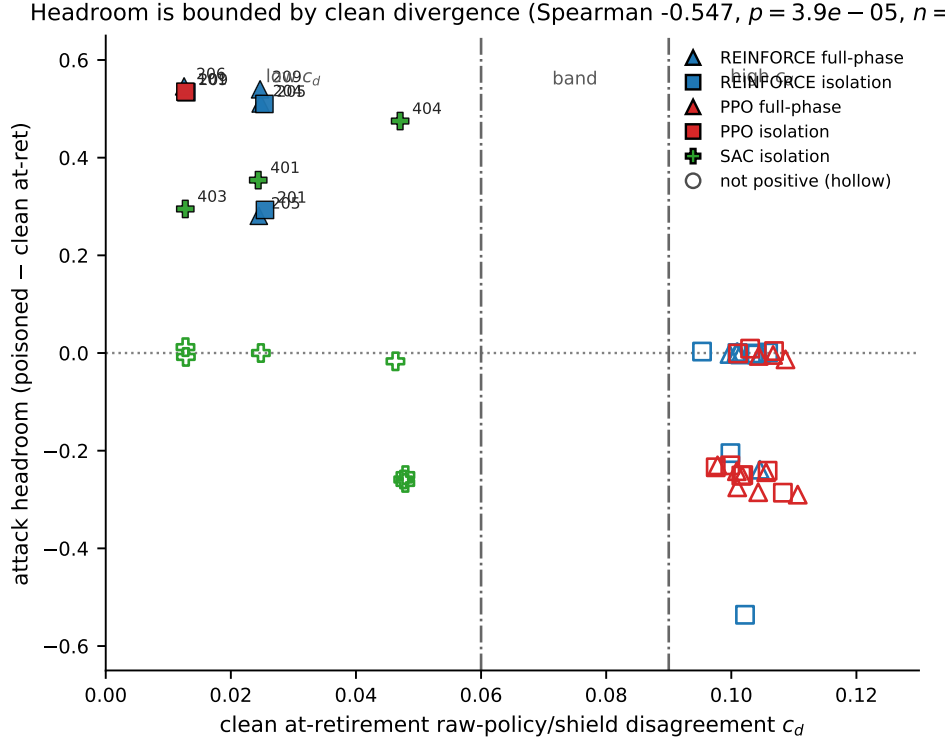


Figure 3: Clean at-retirement raw-policy/shield disagreement marks an opportunity region across the obstacle corpus (both scopes, all three learner families). Each point is one paired clean/poisoned run; filled markers meet the positive-outcome rule, and hollow markers do not. Filled markers all have clean disagreement ≤ 0.047 ; the dashed lines mark the fixed low/high disagreement bands (< 0.06 and ≥ 0.09). No high-disagreement run is positive, while seven of 18 low-disagreement runs are non-positive.

step resident kernel as a negative control and a horizon sweep to bound the required lookahead. Cross-system, detectability, and runtime-cost studies then characterize the scope and operational consequences of the fixed attack. Appendix G re-

ports the low-dimensional semantic benchmarks that separate current-action filtering from update admission.

6.1 Prospective Contract-Separation Audit

All results in this subsection assume the reward-ingestion interface in Table 1. Development grids selected target (18, −5) using five-step acceptance and 120-step harm. Before the sequential V3 audit, we fixed the target, learner, state pools, stop criterion, and bounded perturbation $2 \tanh(\epsilon_t(\mu_t^* - \mu_t))$ on the development run indexed by seed 2070; runs 2071 and 2072 were then evaluated under the unchanged protocol.

All 72 poisoned snapshot–state pairs pass the initial five-step check. Poisoned permanent raw release then failed on 11, 5, and 7 pairs across the three training runs, versus paired clean counts 0, 0, and 2: 21 of 23 poisoned failures are poison-only, two shared, none clean-only (two-sided exact paired $p = 9.537 \times 10^{-7}$), and the CasADi audit identifies the same 23 failures. Resident predictive authority incurs 0/72 failures and switches four to five steps before each corresponding poisoned raw-release failure.

The frozen protocol was then evaluated once on five new learner/evaluation pairs. The target, attacker, learner, budget, state selection, and contracts remained fixed. All 120 snapshot–state pairs passed the initial five-step check. Poisoned raw release failed on 5, 4, 7, 2, and 9 pairs across the five training runs; the paired clean counts were 2, 0, 0, 0, and 0. Resident predictive authority incurred 0/120 failures and switched before all 27 poisoned failures. The 25 poison-only and zero clean-only discordances gave a two-sided exact $p = 5.960 \times 10^{-8}$. Poisoned failure occurred in each of the five independently trained runs.

The gap without an attacker. Two of the 120 clean pairs failed under permanent raw release after passing the same five-step check that accepted them. Two in 120 is a small rate, and the paired design was built to detect the poisoning effect rather than to estimate the clean one, so we do not claim that unattacked release is unsafe in general. We claim what the two failures witness: acceptance and failure are separated here only by horizon, with no adversary in between, and the reward-record channel widens a gap that is already present.

State–snapshot pairs that share a trained learner are not independent training replicates, so the V3 and V4 exact tests summarize within-snapshot discordance; replication across training realizations is reported through the three V3 and five V4 runs. For independent replication and the one-step negative control, snapshots from run indices 2040–2042 used disjoint evaluation pools 8040–8042; 71 of 72 states passed the initial check, and all mechanisms shared state–snapshot keys.

Table 3 makes the three evidence blocks and deployment contracts explicit.

How much lookahead a retained check needs. A locked sweep varies only the retained predictor’s horizon on the pooled V3 cohort of 72 pairs, holding attack, snapshots, and deployment states fixed. Three quantities move together. Release admission falls as the horizon grows: 72, 72, 72, 57,

Table 3: Contract separation after initial five-step acceptance under bounded reward-record write access. V4 is the five-run confirmation; V3 contains the development and sequential audits. The independent audit includes the one-step negative control. A switch transfers control to the trusted LQR.

Contract	Failures	Authority event
<i>V4 confirmation (120 pairs; 5 training runs)</i>		
Clean permanent raw release	2/120	—
Poisoned permanent raw release	27/120	—
Resident predictive authority	0/120	27 switches
<i>Development/sequential V3 (72 paired states)</i>		
Clean permanent raw release	2/72	—
Poisoned permanent raw release	23/72	—
Resident predictive authority	0/72	23 switches
<i>Independent audit (71 accepted states)</i>		
Permanent raw release	19/71	—
One-step resident kernel	14/71	79 int.; 41 empty
Resident predictive authority	0/71	19 switches

and 49 of 72 candidates accepted at horizons 1, 3, 5, 10, and 20. Switching lead time grows: the median switch preceded the corresponding raw-release failure by 0, 2, 4, and 9 steps at horizons 1, 3, 5, and 10. Resident failures track lead time rather than residence: 12 at horizon 1, where switches occurred with zero lead, and 0 at horizons 3, 5, and 10. Horizon 1 failed the predeclared separation condition for insufficient lookahead; horizon 20 failed it from the opposite side, rejecting so many candidates that none of the 49 admitted pairs retained a poisoned failure to separate. The usable lookahead is an interval, and its lower edge is set by the lead time the trusted baseline needs, not by the accuracy of the prediction.

In the independent audit, permanent raw release failed on 6/24, 3/23, and 10/24 accepted pairs. Resident predictive authority repeats the same five-step raw-policy prediction at every physical step and switches on exactly those 19 paired states, four steps before failure; no physical failure remained. In contrast, the one-step resident kernel failed on 14 pairs despite 79 interventions and reached an empty kernel 41 times. Repeated five-step prediction therefore provided an effective recoverability trigger in this study. One-time authorization and one-step filtering did not provide the same protection.

6.2 Reward-Influence Geometry and Optimizer Boundary

We validated the reward-influence analysis on an unclipped cartpole batch from the learner indexed by seed 2040. Reconstructing the implemented gradient from Equation (24) has error 2.78×10^{-17} , and across 22 release-gate halfspaces the largest discrepancy between an SOCP witness and its computed support is 1.07×10^{-9} . Audited on all 36 poisoned V3

batches, the homogenized clipped formulation yields verified support witnesses on 22/36, where the earlier premises admitted 0/36. Exact batch support nevertheless did not explain the successful multi-batch attack: the physical experiment evaluates the fixed target and bounded attacker without establishing joint gate optimality of the tanh rule. Appendix D reports the full implementation audit and its parameter-clipping boundary.

6.3 Cross-System Reward-Record Poisoning

We next tested the lifecycle failure with updates produced by an online learner: a Gaussian residual actor on a trusted controller, with standard reward-to-go REINFORCE gradients computed from PyBullet transitions. A backup action filter protects the adaptation trajectory, and deployment then audits the raw learned snapshot, the object requiring lifecycle admission. Training, certificate, calibration, and deployment states used disjoint run indices; all mechanisms within a training realization shared paired state sets; and attack directions and hyperparameters were fixed on a development realization before the three-run audits. The exact tests operate at the paired deployment-state-block level conditional on the three learned snapshots, so their p -values summarize within-snapshot discordance, and Wilson and paired-block bootstrap intervals have the same conditional scope.

On cartpole, the clean learner violates 1/188 admitted roll-outs and the poisoned action-only learner violates 38/188. There are 37 paired poison-only failures and no clean-only failures (two-sided exact paired $p = 1.455 \times 10^{-11}$). Both commit admission and always-freeze have zero observed violations.

On two-dimensional quadrotor, clean learning has 0/72 violations, while reward-only poisoning causes 68/72, all delayed, across all three training realizations. The paired discordance is 68 poison-only to zero clean-only (two-sided exact $p = 6.776 \times 10^{-21}$). Commit admission and always-freeze again have zero observed violations. The gate retains nonzero fractions 0.55, 0.40, and 0.70 of the three poisoned pending updates. These fractions establish update availability under attack, not benign learning utility.

Learned-baseline variant. Replacing REINFORCE’s constant batch-mean baseline with a learned per-state linear value baseline, under a protocol fixed before execution and retaining the same Monte Carlo return, residual actor, score function, and reward-record channel, poisoning caused 40/126 raw-release failures across three independent training runs, against 2/126 for paired clean release, with commit admission and always-freeze at 0/126 (38 poison-only, zero clean-only, two-sided exact $p = 7.3 \times 10^{-12}$). This is a robustness check within the Monte Carlo policy-gradient family, not a distinct learner family: off-policy and bootstrapped learners change the attack channel through a temporal-difference target, so the official-PPO audit below remains the learner boundary. These

rates use the full 42-state envelope ($126 = 3 \times 42$) without initial-acceptance filtering and are not directly comparable to the filtered 120-pair confirmation cohort.

Non-singleton adaptation set. Adding the observation-FDI channel of Section 3 to the reward-record attacker (a pole-angle bias of radius ρ , so the protective kernel ranges over a box of half-width ρ , about $\pm 2.3^\circ$ at $\rho = 0.04$) left the release certificate and deployment states unchanged and preserved the contract separation at every non-singleton radius. Poisoned raw release failed on 44/126 and 61/126 pairs at $\rho = 0.02$ and 0.04, against 4/126 and 3/126 for clean release, with commit admission and always-freeze at 0/126; the near-singleton reference at $\rho = 0.005$ was 41/126 versus 5/126. Failure counts were non-monotone across training realizations, while the kernel remained nonempty throughout the grid.

6.4 Reward-Record Integrity and Detectability Boundary

We evaluated attack detectability separately from deployment failure. Four checks were fixed on the clean development run indexed by seed 2070. The frozen checks were then evaluated on 84 clean and 84 poisoned batches from seven independent training runs.

Table 5 shows that trusted recomputation blocks every evaluated edit, and even task-specific scalar checks detect over 90% of poisoned batches. The ineffective batch-mean envelope shows why a generic “monitor the average” statement is not enough: detection depends on reward semantics and trusted inputs. Cryptographic origin binding would also reject post-producer mutation if the trusted producer signs the reward together with its transition, action, and time. That producer remains within the trusted computing base of the evaluated threat model.

Temporal and moment-preserving edits tested on the development realization satisfied their integrity constraints and moved parameters toward the harmful target, yet each caused 0/24 raw-release failures. The effective fixed attack was therefore readily detectable by the task-aware checks. Authentication and trusted reward recomputation provide upstream controls; commit admission and resident predictive authority provide downstream containment.

In-loop known-sign gate. We placed the same known-sign check inside the update loop and froze batches that violated the nonpositive reward guarantee. Across three independent training runs, this gate reduced poisoned raw-release failures from 39/126 to 0/126 and froze 35 of 36 poisoned batches. The paired defended-versus-undefended test gave $p = 3.6 \times 10^{-12}$. The gate froze no clean batch but approached always-freeze availability under attack. It detects the sign-violating effective attack, while the sign-respecting variants evade the check and independently caused no deployment harm. This gate therefore implements the measured

Table 4: Paired reward-record poisoning and snapshot admission. “Commit” denotes finite-horizon admission; permanent mechanisms use the common-state audit in Fig. 4. Raw-snapshot rewards after failure are descriptive, not utility comparisons.

System	Mechanism	Learner seeds	Rollouts	Violations	Mean reward
Cartpole	Clean REINFORCE	3	188	1	−0.197
Cartpole	Poisoned action-only	3	188	38	−1.269
Cartpole	Poisoned commit	3	188	0	−0.102
Cartpole	Always-freeze	3	188	0	−0.117
Quadrotor	Clean REINFORCE	3	72	0	−0.030
Quadrotor	Poisoned action-only	3	72	68	−0.170
Quadrotor	Poisoned commit	3	72	0	−0.060
Quadrotor	Always-freeze	3	72	0	−0.029

Table 5: Frozen reward-integrity audit. Rates are batch-level true- and false-positive rates (TPR/FPR) across seven training runs. Each check requires the stated defender assumption.

Check	Required trusted information	TPR/FPR
Reward recomputation	Authenticated transition and deterministic reward	1.000/0
Known-sign check	Task guarantee that reward is nonpositive	0.905/0
Scalar envelope	Clean development range transfers	0.917/0.036
Batch-mean envelope	Clean batch mean transfers	0.012/0.036

detectability boundary; provenance and recomputation provide the broader upstream controls.

6.5 Runtime Escape Conditions and Their Cost

Permanent runtime filtering provides a second architectural response. Its certificate covers the composed policy $F \circ \pi$, allowing the filter to contain raw-policy failure before physical execution. We compared permanent and snapshot contracts on a result-independent pool of 144 states from three disjoint run indices. For each training realization, the common operating set contained states certified for both freeze and the committed snapshot at horizon 100 and guard 0.003. Eight order-spanning states were evaluated from identical state/reset pairs. Each mechanism therefore had 24 paired rollouts and 2400 possible steps.

Table 10 and Fig. 4 show the resulting separation. The poisoned raw snapshots violate 23/24 rollouts and complete only one. Clean, commit, freeze, the permanent five-step backup shield, and Safe-Control-Gym’s official linear model predictive safety certification (MPSC) mechanism all have zero observed failures and complete 24/24. The shield intervenes on 511/2400 steps with no rejection, while MPSC intervenes on 2400/2400 with no infeasible solve. Median/p95 online latency is 221.925/238.216 ms for the shield and 59.657/76.055 ms for MPSC, compared with 0.026–0.027/0.028–0.029 ms

for raw snapshot evaluation. These values are single-process wall-clock measurements rather than worst-case execution-time guarantees.

The safe mechanisms differ in utility. Mean reward was −0.023 for freeze, −0.024 for clean, −0.056 for commit, −0.195 for the shield, and −0.457 for MPSC; paired rollout deltas relative to freeze were −0.00096, −0.03350, −0.17208, and −0.43393, with paired-block bootstrap 95% intervals excluding zero in every case and no improvement over freeze in any of 24 blocks. These values characterize the attack-evaluation frontier; the benign adaptation audit below evaluates learning utility separately.

Table 7 in Appendix E makes the snapshot-admission cost explicit: common operating-set coverage is 99.3%, 100.0%, and 90.3% for the training runs indexed by 2040–2042, and reconstructing trusted-state admission with 21-point contiguous backtracking took 8.49–9.32 s per run, exactly reproducing the locked commit fractions 0.55, 0.40, and 0.70. Offline commit therefore avoids permanent per-action computation but pays finite certificate coverage and update latency. The comparison distinguishes three contracts: certify a snapshot before raw deployment, retain a filter permanently, or stop adapting.

6.6 Boundaries and Validity

Three locked audits bound the result. A trusted state anchor shrinks certificate uncertainty without sanitizing the reward record: every anchor period retains all 16 certificate centers, has 0/24 violations on new paired blocks, and improves no reward over freeze. Against the official pretrained PPO policy, which is safe on 72/72 held-out rollouts, the locked $L_\infty \leq 0.5$ budget produced 1/24 poisoned failures at one checkpoint and 0/24 at another, so the audit showed checkpoint-level update redirection without a stable deep-PPO unsafe-policy effect. Under a benign 0.003 N differential actuator bias, freeze, clean adaptation, and LifecycleGate were all violation-free, and LifecycleGate accepted a nonzero update in 11/12 batches but improves no paired rollout over freeze, so certificate acceptance did not establish useful adaptation. The CasADi gates use fixed finite state sets while physical outcomes use

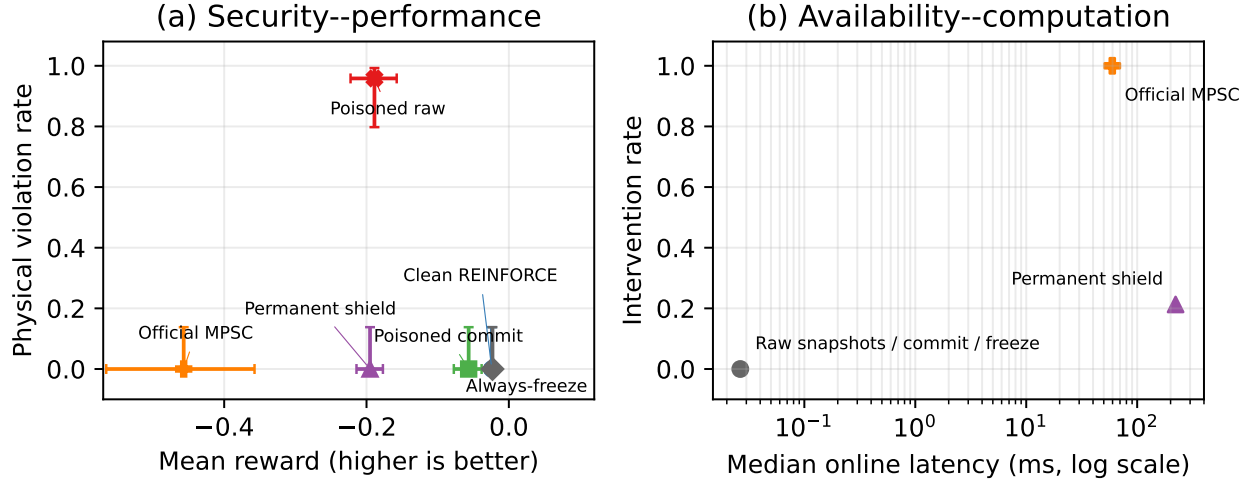


Figure 4: The unified frontier on the same paired state blocks. Error bars in (a) are 95% Wilson intervals for violation rate and paired-block bootstrap intervals for mean rollout reward, conditional on the three locked learner snapshots. Panel (b) separates the negligible raw policy latency from the permanently-online protection cost; raw, commit, and freeze mechanisms overlap near 0.026 ms and zero intervention.

independent PyBullet rollouts; they are sampled finite-horizon certificates, not continuous-state invariant-set proofs. Realized state violations and pre-clip saturation are reported separately. Appendix E gives the full figures.

7 Related Work

Runtime assurance and safe controller updates. Simplex architectures retain a decision module and a trusted baseline during online upgrades [1]; neural and barrier-based Simplex extend this with online retraining, reverse switching, and switching conditions under bounded model error [8, 9]; and model-predictive safety certification (MPSC) and runtime shields preserve recoverability or prevent unsafe actions [2, 3, 17]. All of these certify a resident composition, whereas we consider a transition in which the raw policy later receives authority. Policy repair, safe projection, joint controller/certificate repair, robust conformal transfer, and shielded sim-to-real transfer instead establish evidence for the deployment object before use [5, 10–12, 18, 19].

Shield retirement and attacked state semantics. Carr et al. [4] study a partially observable bootstrap-to-retirement lifecycle and compare sudden with smooth shield removal under trusted learning signals. We reuse that published lifecycle as a third-party case study and add bounded reward-record poisoning during the protected bootstrap; the resulting experiment concerns evidence reuse across the same authority transition, not the soundness of their original claims.

Secure estimation characterizes when sensor attacks can be detected or corrected [14, 20], and severe-attack CBF filtering constructs plausible-state sets and safe-action kernels when the state cannot be reconstructed uniquely [13]. Our model consumes such a set and asks whether an adversari-

Table 6: Closest work along the three axes combined in this paper: adversarial online update data, removal of runtime-assurance authority, and independent evidence for the released raw policy.

Work	Adversarial update log	Authority removed	Independent raw-policy evidence
Simplex / Neural Simplex [1, 8]	no	no (monitor resident)	n/a (composition certified)
Bb-Simplex [9]	no (trusted re-training)	no	n/a
Policy repair / MPSC [3, 10]	no (trusted evidence)	repaired, verified	repaired
Hsu et al. [5]	no	yes	yes (transfer evidence)
Carr et al. [4]	no	yes (sudden/smooth)	empirical retirement
BADControl [27]	operational data, physical trigger	n/a	n/a
Controlled study	yes (reward record)	yes (raw release)	transition evidence tested
Third-party case study	yes (reward record)	yes (Carr retirement)	transition evidence tested

ally updated action map remains inside its certified kernel. Related lines learn barriers, certify reinforcement-learning robustness, or solve adversarial reach-avoid problems under trusted learning signals [21–26].

Policy and reward poisoning. Policy poisoning has been studied in batch reinforcement learning and control [28]; online attacks perturb individual rewards under an ℓ_∞ budget and adapt to the learning process [29]; and extensions cover policy-gradient learners without known dynamics and black-box deep reinforcement learning, and targeted poisoning can teach a chosen policy [30–33]. None of this line places the learner behind a runtime shield.

BADControl poisons operational data in PID and LQR loops to implant a vulnerability activated by a physical trigger [27]. Our contribution concerns a different composition: bounded reward-record poisoning affects a policy trained behind runtime assurance, and failure appears after authority retirement without an activation input. Where online reward-poisoning defenses model adaptive attacker–defender interaction [34], we separate provenance, trusted recomputation, detection, commit admission, and runtime containment.

Conic reward-influence analysis. Appendix C applies classical homogenization and second-order-cone feasibility [35–38] to the normalized image of a centered reward-to-go zonotope.

8 Discussion and Limitations

The evidence supports an architectural vulnerability with conditional exploitability. Protected adaptation certifies the executed composition while resident authority remains active; permanent raw release changes the subject, authority, provenance, information, and horizon of that evidence. Neither result requires the shield or filter itself to be unsound, and the clean cohort fixes the ordering: the transition is a contract gap before it is an attack surface, and the upstream channel changes the rate of post-release failure rather than creating its possibility.

The attack has a structured opportunity region that does not determine success: every positive outcome in the locked corpus occurred at low retirement-state disagreement and no high-disagreement run was positive, yet the fresh REINFORCE pair, seven low-band corpus runs, and the SAC battery show that headroom is insufficient for the fixed attack. The defenses separate integrity from authority. Provenance binding and trusted reward recomputation remove the evaluated upstream channel while leaving the contract gap in place; task-aware checks detect the effective fixed attack under additional semantic assumptions; commit admission limits which raw snapshot receives authority; and resident predictive authority contains unsafe proposals at runtime, where the one-step kernel shows that residence alone is insufficient and the decision rule must preserve recoverability over the relevant horizon. These contracts incur different coverage, intervention, latency, and reward costs, so the choice is an explicit architectural tradeoff rather than a default.

8.1 Limitations

The marker is descriptive, not an attacker trigger. Retirement-state disagreement is observed after the protected bootstrap and therefore characterizes a realized deployment state. An attacker-side trigger would require evidence that earlier disagreement trajectories predict the eventual opportunity region. The reported AUC is consequently a descriptive measure over the locked corpus rather than an externally validated classifier,

and the marker is obstacle-scoped: the second environment also contained protective outcomes at high clean disagreement.

The formal results have bounded verification scope. The current- and future-history gates state certificate obligations, and the linear parameter gate is exact on its finite abstraction; the continuous lift additionally requires a justified reachable-set cover, regularity and model-error bounds, and a robust margin, which the coverage audit probes without establishing continuous-state invariance. The exact reward-influence result covers a fixed standardized-advantage batch with global gradient clipping, leaving coordinatewise parameter clipping and multi-batch attack construction outside its scope.

The empirical generality is bounded. The end-to-end physical effect is established for low-dimensional Monte Carlo policy-gradient learners. The learned-baseline variant shares the same return and attack channel rather than forming an independent learner family. The official Safe-Control-Gym PPO audit showed checkpoint-level redirection without stable physical failure. On the third-party lifecycle, PPO and SAC contained individual positive training realizations but did not provide reliable success across paired runs.

The threat model starts after access is obtained. It does not estimate the prevalence of bounded reward-ingestion access across products, and it does not model service exploitation. The effective fixed attack is detectable by the tested task-aware checks, and the sign-respecting variants did not retain deployment harm. The trusted-anchor audit models bounded error only for altitude and pitch and reports calls rather than hardware-specific cost. These boundaries limit claims about stealth, cross-architecture prevalence, and deployment overhead without changing the authority-transition result.

9 Conclusion

A release test is not runtime authority. Protected adaptation and permanent raw release require different security evidence, and the finite check that authorizes the transition is the evidence that does not survive it. Permanent raw release failed on 2/120 snapshot–state pairs with a clean learner and 27/120 with a poisoned one, all after the same five-step check accepted them; resident predictive authority failed on none, a one-step resident kernel still failed on 14/71, and the horizon sweep bounded the useful lookahead from both sides. On Carr et al.’s published shield-retirement workflow, bootstrap-only poisoning increased raw-policy violations at retirement by factors of 3.6 and 1.6 in two of three provenance-complete paired REINFORCE runs, while protected execution stayed violation-free; exploitability remained conditional on the realized retirement state. Provenance binding and trusted recomputation close that channel but not the gap it exploits: evidence collected for an assured composition stays bound to its subject, authority, provenance, information, and horizon, and raw release must re-establish each obligation it changes.

Ethical Considerations

This paper studies an attack on safety-critical learning-enabled control. All experiments run in simulation using Safe-Control-Gym with PyBullet dynamics and a public third-party gridworld implementation. No physical plant, vehicle, deployed industrial system, human subject, or personal data was involved. Reported failures are simulated constraint violations used to compare deployment contracts.

The evaluated threat begins *after* compromise of a reward-ingestion principal in the asynchronous learning-data plane and grants only bounded scalar writes to reward records. Initial access, product-specific exploitation, and evasion of log-integrity controls fall outside the artifact and evaluation. The experiments characterize the consequence of a least-privilege data-integrity gap for evidence inheritance at raw-policy release. We therefore describe a composition weakness in an assurance workflow rather than an exploit against an identified fielded product, and we publish no capability that lowers the cost of obtaining the assumed access.

Stakeholders are operators of runtime-assured learning controllers, vendors of shielding and safety-filter tooling, and the public exposed to the controlled plant. The main foreseeable harm from publication is that an attacker who already holds reward-ingestion access learns that a retired shield leaves latent policy damage exposed. We judge the disclosure benefit to dominate: the exposure follows from documented assurance architectures, the attack is detected by the simple task-aware reward checks we measure, and the paper states deployable countermeasures at both ends of the authority transition. Origin-bound reward records and trusted recomputation remove the upstream precondition; resident predictive authority, commit admission, and permanent filtering contain the downstream physical consequence.

The third-party lifecycle we instrument is a published research artifact released under an open license by its authors, used unmodified except for documented instrumentation and reward-record patches that the artifact itemizes. No vendor product, hosted service, or operational deployment was tested, so no coordinated vulnerability disclosure was owed to a third party. We follow the USENIX Security ethics guidelines and the principles of the Menlo Report in reporting the above.

Open Science

We support the USENIX Security open science policy and will make every artifact required to reproduce the reported results openly available. The artifact contains: (i) the experiment drivers for the controlled Safe-Control-Gym study and the third-party shield-retirement case study; (ii) row-level result files backing every table and figure, with source-linked provenance columns; (iii) exact commands, run namespaces, upstream revisions, model files, and dependency versions; (iv) locked protocol documents recording the analysis chronology;

and (v) SHA-256 checksums for all locked files. Automated checks in the artifact audit table provenance, claim locks, and manuscript format.

For anonymous review, the artifact is submitted as an anonymized archive through the submission system, so the review committees can access every component without contacting the authors. Upon acceptance we will publish the same artifact in a public repository with a permanent DOI under an open license, together with the third-party upstream revision pins required to rebuild the case-study environment. The artifact retains the scope boundaries stated in the paper: the trusted-anchor, certificate-coverage, timing, PPO, benign-utility, and protocol-chronology limitations are recorded with the data rather than removed.

References

- [1] D. Seto, B. Krogh, L. Sha, and A. Chutinan, “The simplex architecture for safe on-line control system upgrades,” in *Proceedings of the 1998 American Control Conference*, vol. 6, 1998, pp. 3504–3508.
- [2] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [3] K. P. Wabersich and M. N. Zeilinger, “Linear model predictive safety certification for learning-based control,” in *2018 IEEE Conference on Decision and Control*, 2018, pp. 7130–7135.
- [4] S. Carr, N. Jansen, S. Junges, and U. Topcu, “Safe reinforcement learning via shielding under partial observability,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 12, 2023, pp. 14 748–14 756.
- [5] K.-C. Hsu, A. Z. Ren, D. P. Nguyen, A. Majumdar, and J. F. Fisac, “Sim-to-lab-to-real: Safe reinforcement learning with shielding and generalization guarantees,” *Artificial Intelligence*, p. 103811, 2023.
- [6] K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, S. Saravia, A. Sherule, and M. Thompson, “Guide to operational technology (OT) security,” National Institute of Standards and Technology, Tech. Rep. Special Publication 800-82 Revision 3, 2023.
- [7] MITRE ATT&CK, “Data historian, asset a0006,” <https://attack.mitre.org/assets/A0006/>, 2023, accessed 2026-08-05.
- [8] D. T. Phan, R. Grosu, N. Jansen, N. Paoletti, S. A. Smolka, and S. D. Stoller, “Neural simplex architecture,” in *NASA Formal Methods*, ser. Lecture Notes in Computer Science, vol. 12229. Springer, 2020, pp. 97–114.
- [9] A. Damare, S. Roy, R. Sharma, K. DSouza, S. A. Smolka, and S. D. Stoller, “A barrier certificate-based simplex architecture for systems with approximate and hybrid dynamics,” *IEEE Access*, vol. 13, pp. 157 742–157 763, 2025.
- [10] W. Zhou, R. Gao, B. Kim, E. Kang, and W. Li, “Runtime-safety-guided policy repair,” in *Runtime Verification*, ser. Lecture Notes in Computer Science, vol. 12399. Springer, 2020, pp. 131–150.
- [11] P. Lu, M. Cleaveland, O. Sokolsky, I. Lee, and I. Ruchkin, “Repairing learning-enabled controllers while preserving what works,” in *2024 ACM/IEEE 15th International Conference on Cyber-Physical Systems*. IEEE, 2024, pp. 1–11.
- [12] Y. Chow, O. Nachum, A. Faust, E. Dueñez-Guzman, and M. Ghavamzadeh, “Safe policy learning for continuous control,” in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 155. PMLR, 2021, pp. 801–821. [Online]. Available: <https://proceedings.mlr.press/v155/chow21a.html>

- [13] X. Tan, P. Ong, P. Tabuada, and A. D. Ames, “Safety of linear systems under severe sensor attacks,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.08413>
- [14] H. Fawzi, P. Tabuada, and S. Diggavi, “Secure estimation and control for cyber-physical systems under adversarial attacks,” *IEEE Transactions on Automatic Control*, vol. 59, no. 6, pp. 1454–1467, 2014.
- [15] H. Zhang, Z. Li, and A. Clark, “Safe control for nonlinear systems under faults and attacks via control barrier functions,” 2022. [Online]. Available: <https://arxiv.org/abs/2207.05146>
- [16] Z. Yuan, A. W. Hall, S. Zhou, L. Brunke, M. Greeff, J. Panerati, and A. P. Schoellig, “Safe-control-gym: A unified benchmark suite for safe learning-based control and reinforcement learning in robotics,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11 142–11 149, 2022.
- [17] A. Didier, K. P. Wabersich, and M. N. Zeilinger, “Adaptive model predictive safety certification for learning-based control,” in *2021 60th IEEE Conference on Decision and Control*, 2021, pp. 809–815.
- [18] E. Yu, D. Žikelić, and T. A. Henzinger, “Neural control and certificate repair via runtime monitoring,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 25, 2025, pp. 26 409–26 417.
- [19] O. Mirzaeododangeh, E. S. Shekhtman, N. Matni, and L. Lindemann, “Safe planning in interactive environments via iterative policy updates and adversarially robust conformal prediction,” in *Proceedings of The 8th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 331. PMLR, 2026, pp. 678–704. [Online]. Available: <https://proceedings.mlr.press/v331/mirzaeododangeh26a.html>
- [20] F. Pasqualetti, F. Dörfler, and F. Bullo, “Attack detection and identification in cyber-physical systems,” *IEEE Transactions on Automatic Control*, vol. 58, no. 11, pp. 2715–2729, 2013.
- [21] L. Brunke, S. Zhou, and A. P. Schoellig, “Barrier bayesian linear regression: Online learning of control barrier conditions for safety-critical control of uncertain systems,” in *Proceedings of the 4th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 168. PMLR, 2022, pp. 881–892. [Online]. Available: <https://proceedings.mlr.press/v168/brunke22a.html>
- [22] A. Capone, R. K. Cosner, A. Ames, and S. Hirche, “Learning safe control via on-the-fly bandit exploration,” in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 6726–6745. [Online]. Available: <https://proceedings.mlr.press/v267/capone25a.html>
- [23] J. Wu, H. Sibai, and Y. Vorobeychik, “Certifying safety in reinforcement learning under adversarial perturbation attacks,” 2022. [Online]. Available: <https://arxiv.org/abs/2212.14115>
- [24] L. H. Pham and J. Sun, “Certified continual learning for neural network regression,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.06697>
- [25] K.-C. Hsu, D. P. Nguyen, and J. F. Fisac, “ISAACS: Iterative soft adversarial actor-critic for safety,” in *Proceedings of The 5th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 211. PMLR, 2023, pp. 90–103. [Online]. Available: <https://proceedings.mlr.press/v211/hsu23a.html>
- [26] D. P. Nguyen, K.-C. Hsu, W. Yu, J. Tan, and J. F. Fisac, “Gameplay filters: Safe robot walking through adversarial imagination,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.00846>
- [27] L. Burbano, H. Sasahara, R. Song, Z. B. Celik, and A. A. Cárdenas, “BADControl: Backdoor attacks against control systems,” in *Proceedings of the 35th USENIX Security Symposium*, 2026. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity26/presentation/burbano>
- [28] Y. Ma, X. Zhang, W. Sun, and X. Zhu, “Policy poisoning in batch reinforcement learning and control,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 14 543–14 553.
- [29] X. Zhang, Y. Ma, A. Singla, and X. Zhu, “Adaptive reward-poisoning attacks against reinforcement learning,” in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 11 225–11 234. [Online]. Available: <https://proceedings.mlr.press/v119/zhang20u.html>
- [30] Y. Sun, D. Huo, and F. Huang, “Vulnerability-aware poisoning mechanism for online RL with unknown dynamics,” in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=9r30XCjf5Dt>
- [31] Y. Xu, Q. Zeng, and G. Singh, “Efficient reward poisoning attacks on online deep reinforcement learning,” *Transactions on Machine Learning Research*, 2025. [Online]. Available: <https://openreview.net/forum?id=25G63IDHV2>
- [32] A. Rakhsha, G. Radanovic, R. Devidze, X. Zhu, and A. Singla, “Policy teaching via environment poisoning: Training-time adversarial attacks against reinforcement learning,” in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 7974–7984. [Online]. Available: <https://proceedings.mlr.press/v119/rakhsha20a.html>
- [33] T. Everitt, V. Krakovna, L. Orseau, and S. Legg, “Reinforcement learning with a corrupted reward channel,” in *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017, pp. 4705–4713.
- [34] A. Nika, A. Singla, and G. Radanovic, “Online defense strategies for reinforcement learning against adaptive reward poisoning,” in *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 206. PMLR, 2023, pp. 335–358. [Online]. Available: <https://proceedings.mlr.press/v206/nika23a.html>
- [35] S. Schaible, “Duality in fractional programming: A unified approach,” *Operations Research*, vol. 24, no. 3, pp. 452–461, 1976.
- [36] A. Agrawal and S. Boyd, “Disciplined quasiconvex programming,” *Optimization Letters*, vol. 14, no. 7, pp. 1643–1657, 2020.
- [37] H. H. Bauschke, T. Bendit, and H. Wang, “The homogenization cone: Polar cone and projection,” *Set-Valued and Variational Analysis*, vol. 31, no. 3, 2023.
- [38] A. Belloni and R. M. Freund, “On the second-order feasibility cone: Primal-dual representation and efficient projection,” *SIAM Journal on Optimization*, vol. 19, no. 3, pp. 1073–1092, 2008.

A Deferred Proofs

Proof of Proposition 1. Each component is a premise of the assurance decision. A conclusion derived from one premise set is not preserved after a premise is removed or weakened. In the evaluated permanent-release transition, the subject changes from the composition to the raw policy and resident authority is removed. The update provenance becomes adversarial, the policy acts on deployment observations, and a finite check authorizes an unmonitored future. The original evidence therefore cannot be reused without re-establishing these obligations. $\square$

Proof of Lemma 1. The certified action must satisfy the certificate for every state compatible with the attacked history.

These states form $X_{\mathcal{A}}(h)$, and their common certified actions form $K_{\Gamma}(h)$. Continued certified operation therefore requires $\pi_{\theta_{t+1}}(h) \in K_{\Gamma}(h)$. An empty kernel requires rejection, freeze, rollback, or trusted information that changes the plausible-state set. $\square$

Proposition 3 (Action filtering is not update certification). *Let $F(v, h)$ be an action filter that returns an applied input in $K_{\Gamma}(h)$ whenever the kernel is nonempty. Consider a mechanism that first computes $\theta^+ = U(\theta_t, h)$ and then applies*

$$u_t = F(\pi_{\theta^+}(h), h), \quad (10)$$

The mechanism accepts the update without constraining the raw updated policy π_{θ^+} . If $\pi_{\theta^+}(h) \notin K_{\Gamma}(h)$, the applied input can be certified while the updated learner remains uncertified. Action-only filtering therefore does not imply Lemma 1.

Proof of Proposition 3. The filter property gives $u_t \in K_{\Gamma}(h)$, so the current actuator input can satisfy the certificate. Lemma 1 instead constrains the updated controller through $\pi_{\theta^+}(h) \in K_{\Gamma}(h)$. When this membership fails, the certificate covers only the filtered actuator channel. Accepting the update under that certificate creates a stale certified policy. $\square$

Proof of Theorem 1. A lifecycle certificate covers every attacked history reachable under the updated controller and admissible attack family. Applying Lemma 1 pointwise gives $\pi_{\theta^+}(h') \in K_{\Gamma}(h')$ for each such history. This is the definition of $\theta^+ \in \Theta_{\Gamma}(h)$. A failed membership leaves the updated action map uncertified at that history. An empty kernel additionally requires a changed information set or rejection of certified operation. $\square$

Proof of Proposition 2. For fixed z , membership $\pi_{\theta}(z) \in [\ell_z, u_z]$ is equivalent to the two inequalities above. Enforcing them for every $z \in \mathcal{Z}$ implements the lifecycle gate on the abstraction. Parameter box constraints preserve convexity. A feasible or projected candidate satisfies every certified interval kernel in the abstraction. $\square$

Proof of Proposition 5. The standardized advantage vector is $y/\sqrt{T^{-1}\|y\|_2^2} = \sqrt{T}y/\|y\|_2$. Substitution into the usual mean score-function gradient $T^{-1}J^{\top}(\sqrt{T}y/\|y\|_2)$ gives (24). Equation (23) is affine in the reward-box variable, but division by its Euclidean norm is not; hence the normalized image is generally non-affine. $\square$

B Continuous Lift, Kernel Instantiation, and a Minimal Counterexample

B.1 A Margin-Robust Finite-Abstraction Lift

Proposition 2 is exact on its listed grid. Lifting that result to continuous reachable histories requires coverage and regularity assumptions. Let $z = \psi(h')$ be a finite-dimensional

representation on which the updated policy depends. Suppose the fixed-controller certificate can be written as finitely many obligations

$$c_j(x, u, w, x^+) \leq 0, \quad j = 1, \dots, m, \quad (11)$$

where $x^+ = f(x, u, w)$. Invariant, discrete-time barrier, and bounded-horizon reachability checks can be written in this form after augmenting the state with time or certificate memory.

Theorem 2 (Margin-robust finite-abstraction sufficiency). *Fix a candidate snapshot π_{θ} and a future-history horizon. Let $\{(z_i, x_{iq}, w_{ir})\}$ be a finite set such that, for every reachable attacked history h' , every $x \in X_{\mathcal{A}}(h')$, and every $w \in \mathcal{W}$, some sampled triple satisfies*

$$\|z - z_i\| \leq \delta_z, \quad \|x - x_{iq}\| \leq \delta_x, \quad \|w - w_{ir}\| \leq \delta_w, \quad (12)$$

where $z = \psi(h')$. Assume:

1. $\pi_{\theta}(z) \in \mathcal{U}$ and $\|\pi_{\theta}(z) - \pi_{\theta}(z')\| \leq L_{\pi}\|z - z'\|$;
2. the certificate model $\hat{f}$ has uniform error $\|f(x, u, w) - \hat{f}(x, u, w)\| \leq \epsilon_f$;
3. $\hat{f}$ is separately Lipschitz with constants L_f^x, L_f^u, L_f^w ; and
4. each c_j is separately Lipschitz in (x, u, w, x^+) with constants $L_j^x, L_j^u, L_j^w, L_j^+$.

Define

$$\begin{aligned} \eta_j = & (L_j^x + L_j^+ L_f^x) \delta_x \\ & + (L_j^u + L_j^+ L_f^u) L_{\pi} \delta_z \\ & + (L_j^w + L_j^+ L_f^w) \delta_w + L_j^+ \epsilon_f. \end{aligned} \quad (13)$$

If every sampled triple satisfies

$$c_j(x_{iq}, \pi_{\theta}(z_i), w_{ir}, \hat{f}(x_{iq}, \pi_{\theta}(z_i), w_{ir})) \leq -\eta_j \quad (14)$$

for all j , then $\pi_{\theta}(\psi(h')) \in K_{\Gamma}(h')$ for every reachable attacked history h' in the horizon. Hence the snapshot satisfies the continuous future-history lifecycle condition of Theorem 1.

Proof. Fix h' , $x \in X_{\mathcal{A}}(h')$, and $w \in \mathcal{W}$, and select a sampled triple using (12). Write $u = \pi_{\theta}(z)$ and $u_i = \pi_{\theta}(z_i)$. Policy regularity gives $\|u - u_i\| \leq L_{\pi} \delta_z$. The model-error and Lipschitz assumptions give

$$\begin{aligned} \|f(x, u, w) - \hat{f}(x_{iq}, u_i, w_{ir})\| \leq & \epsilon_f + L_f^x \delta_x \\ & + L_f^u L_{\pi} \delta_z + L_f^w \delta_w. \end{aligned} \quad (15)$$

Applying the four Lipschitz bounds of c_j and collecting terms yields

$$\begin{aligned} c_j(x, u, w, f(x, u, w)) \leq & c_j(x_{iq}, u_i, w_{ir}, \\ & \hat{f}(x_{iq}, u_i, w_{ir})) + \eta_j \\ \leq & 0. \end{aligned} \quad (16)$$

The argument holds for every plausible state, disturbance, certificate obligation, and reachable attacked history, which is precisely membership in the corresponding kernels over the stated horizon. $\square$

Theorem 2 converts a justified joint cover into a verification result. Establishing that cover remains a separate obligation; a finite rollout sample is insufficient. The coverage audit in Appendix H provides empirical evidence about this premise without constituting a cover proof.

Proposition 4 (Plausible-set and freeze-frontier monotonicity). *Fix a history domain $\mathcal{H}$, dynamics, certificate obligations, and policy class. If two information contracts satisfy $X_1(h) \subseteq X_2(h)$ for every $h \in \mathcal{H}$, let $K_{\Gamma,k}$ be the kernel induced by X_k and let $\Theta_{\Gamma,k} = \{\theta : \pi_\theta(h) \in K_{\Gamma,k}(h), \forall h \in \mathcal{H}\}$. Then*

$$K_{\Gamma,2}(h) \subseteq K_{\Gamma,1}(h) \quad \text{and} \quad \Theta_{\Gamma,2} \subseteq \Theta_{\Gamma,1}. \quad (17)$$

Consequently, for a nested ambiguity family $X_{\rho_1}(h) \subseteq X_{\rho_2}(h)$ whenever $\rho_1 \leq \rho_2$, certified update feasibility is non-increasing in ρ : once the certified parameter set is empty, it remains empty at larger ambiguity. Conversely, a trusted anchor that pointwise shrinks the plausible-state sets can only weakly enlarge the kernels and certified parameter set.

Proof. An action certified for every state in the larger set $X_2(h)$ is certified for every state in its subset $X_1(h)$, proving kernel inclusion. Requiring a policy action to lie in the smaller kernel $K_{\Gamma,2}(h)$ for every $h \in \mathcal{H}$ is at least as restrictive as requiring membership in $K_{\Gamma,1}(h)$, proving parameter-set inclusion. The frontier and anchor statements follow by applying these inclusions to nested families. $\square$

The monotonicity comparison holds on a fixed history domain and certificate contract. If an anchor changes which histories are reachable, or if two sampled plausible sets are not nested, neither conclusion follows automatically. The P3 anchor audit therefore constructs nested samples explicitly.

The three results have distinct scopes. The future-history theorem states a necessary certificate condition. Proposition 2 is exact on its finite abstraction. Theorem 2 supplies a sufficient lift under its cover, regularity, and margin assumptions.

B.2 Instantiating the Kernel

For an invariant certificate $I \subseteq S$, the kernel is

$$K_I(h) = \{u \in \mathcal{U} : \forall x \in X_{\mathcal{A}}(h), \forall w \in \mathcal{W}, f(x, u, w) \in I\}. \quad (18)$$

For a barrier certificate $B(x) \geq 0$ with safe set $C = \{x : B(x) \geq 0\}$, the corresponding kernel is

$$K_B(h) = \{u \in \mathcal{U} : \forall x \in X_{\mathcal{A}}(h), \forall w \in \mathcal{W}, B(f(x, u, w)) \geq 0\}. \quad (19)$$

A reachability certificate starts from $X_{\mathcal{A}}(h)$ rather than a nominal state estimate. A runtime shield must select an action in

$K_{\Gamma}(h)$. An empty kernel requires rejection, fail-safe operation, or a trusted state anchor.

B.3 A Minimal Counterexample

Consider

$$x_{t+1} = \lambda x_t + u_t, \quad |u_t| \leq u_{\max}, \quad S = [-1, 1]. \quad (20)$$

Suppose FDI makes the same history compatible with $x_t = r$ and $x_t = -r$, where $r \in (0, 1]$. The one-step safe action set for state x is

$$\mathcal{U}_{\text{safe}}(x) = [-u_{\max}, u_{\max}] \cap [-1 - \lambda x, 1 - \lambda x]. \quad (21)$$

For $x = r$ and $x = -r$, these sets are individually nonempty but disjoint whenever

$$1 < \lambda r < 1 + u_{\max}. \quad (22)$$

The strict upper bound makes each known state individually controllable. FDI creates indistinguishability, so no single history-dependent action certifies both states.

This example distinguishes fixed filtering from certified learning. A fixed severe-attack filter can reject when the common kernel is empty. A certified online learner must also prevent updates under a certificate with no feasible kernel.

C Exact Positive Support Under Gradient Clipping

This appendix states and proves the one-batch reward-influence results summarized in Section 4.2.

C.1 Normalized One-Batch Proposal Set

This subsection connects the reward-record attacker to the release halfspaces of Proposition 2. For a fixed on-policy batch of length T , let $J \in \mathbb{R}^{T \times d}$ contain the policy-score row vectors, let $r \in \mathbb{R}^T$ be the logged rewards, let $H_{ij} = \mathbf{1}[j \geq i]$ be the reward-to-go operator, and let $C = I - T^{-1} \mathbf{1} \mathbf{1}^\top$ be the centering matrix. An attacker choosing $\delta \in [-\epsilon, \epsilon]^T$ produces the centered return vector

$$y(\delta) = CH(r + \delta). \quad (23)$$

The fixed-batch assumption is appropriate for a reward-record-only attacker: changing a reward after collection does not change the states, actions, or score rows already in the batch.

Proposition 5 (Standardized reward-influence set). *Suppose the learner standardizes the centered reward-to-go advantages with population standard deviation whenever $\|y(\delta)\|_2 > 0$. Before optimizer clipping, a gradient-ascent proposal of size α is*

$$\tilde{\theta}^+(\delta) = \theta + \frac{\alpha}{\sqrt{T}} J^\top \frac{y(\delta)}{\|y(\delta)\|_2}. \quad (24)$$

Consequently, the raw one-batch proposal set is the image of the centered reward-to-go zonotope under normalization and

the policy-score map; in general, it is not a parameter-space zonotope.

Normalization is why a bounded reward box does not induce a bounded parameter-space zonotope, and it is also what makes the influence question tractable. The support characterization below applies to one fixed batch; later batches change J , r , and the visited states. The implementation skips standardization below a numerical tolerance, so each reported cone witness must remain above that threshold.

C.2 Exact Positive Support

Theorem 3 (Positive support under global gradient clipping). *Let $B = J^\top / \sqrt{T}$, let the learner cap the global gradient norm at $G > 0$, and assume coordinatewise parameter clipping is inactive over the reward box. The post-cap update before parameter clipping is*

$$\theta_G^+(y) = \theta + \frac{\alpha B y}{\max\{\|y\|_2, \|B y\|_2 / G\}}. \quad (25)$$

For a release halfspace $a^\top P \theta \leq b$, define $q = \alpha B^\top P^\top a$, $y_0 = CHr$, and $M = CH$. For any fixed $t > 0$, a reward perturbation whose gate-coordinate increment is at least t exists if and only if the following second-order-cone problem is feasible:

$$\begin{aligned} \lambda &\geq 0, & -\epsilon \lambda \mathbf{1} &\leq v \leq \epsilon \lambda \mathbf{1}, \\ \bar{y} &= \lambda y_0 + M v, & q^\top \bar{y} &= 1, \\ t \|\bar{y}\|_2 &\leq 1, & (t/G) \|B \bar{y}\|_2 &\leq 1. \end{aligned} \quad (26)$$

Feasibility is monotone in t , so bisection on $[0, \min\{\|q\|_2, \alpha G \|P^\top a\|_2\}]$ computes the exact positive support

$$\max \left\{ 0, \sup_{\|\delta\|_\infty \leq \epsilon} \frac{q^\top y(\delta)}{\max\{\|y(\delta)\|_2, \|B y(\delta)\|_2 / G\}} \right\}. \quad (27)$$

The construction remains valid when the centered-return zonotope contains the origin; if $G = \infty$, the second norm constraint is omitted. If this support does not exceed the current halfspace margin, no admissible reward edit can cross that halfspace in the batch.

Proof. Global norm clipping gives the stated maximum in the denominator. The ratio is positively homogeneous, so optimization over the zonotope $Z = \{y_0 + M\delta : \|\delta\|_\infty \leq \epsilon\}$ is equivalent to optimization over its conic hull. The first line of (26) is the perspective representation of that hull: $v = \lambda\delta$. This is the standard homogenization used in fractional and conic optimization [35, 37].

If a positive ratio reaches t , rescale its conic representative until $q^\top \bar{y} = 1$; its two denominator branches are then exactly the last line of (26). Conversely, any feasible $\bar{y}$ is nonzero because of the unit-numerator equality and recovers a reward-box ray whose ratio reaches t . Thus the origin cannot create the degenerate feasibility present in a direct zonotope formulation. Each fixed- t problem is a second-order

feasibility-cone problem [38]; monotonicity permits quasi-convex bisection [36]. Cauchy–Schwarz supplies both stated upper bounds. The margin result follows by adding the maximum increment to the current gate coordinate. $\square$

D Reward-Influence Implementation Audit

This appendix expands the batch-level audit summarized in Section 6.

We first validated the reward-influence analysis on an unclipped cartpole batch from the learner indexed by seed 2040. Reconstructing the implemented gradient from Equation (24) has error 2.78×10^{-17} . The clean gradient norm is 0.196789, neither gradient nor parameter clipping is active, and the minimum centered-return norm over the reward box is 7.993864. Across 22 release-gate halfspaces, the largest discrepancy between an SOCP witness and its computed support is 1.07×10^{-9} . The minimum robust margin was 4.598004, so no halfspace could be crossed in this batch. The successful physical attack instead used the locked bounded tanh target-shaping rule and accumulated influence across changing batches. It was not generated by a one-step second-order cone program (SOCP).

The homogenized clipped formulation is then audited on all 36 poisoned V3 batches. The old premises make 0/36 eligible; the new formulation yields verified support witnesses on 22/36, including 18 whose reward boxes contain a normalization singularity and 20 for which the old global gradient-clipping exclusion fails. Every actual update is below its computed support, all witnesses respect the reward budget, final parameter reconstruction error is zero, and maximum witness/support error is 3.67×10^{-5} . Ten batches cannot globally exclude coordinatewise parameter clipping (nine activate it on the observed edit), and four cone-optimal directions have no recovered witness above the learner’s normalization tolerance; they remain outside the exact implementation audit. The clipped quadrotor trajectories have not received this separate parameter-clipping audit and remain empirical.

Exact batch support did not explain the successful multi-batch attack. The locked tanh update advances in the target direction on only 6/20 eligible batches with positive oracle increment, and its preregistered same-trajectory advantage reaches 22/36 rather than the required 27/36. The original exact-support audit emitted 0/12 edits under the earlier singularity criterion. A diagnostic rerun on the development realization used the revised solver and emitted nonzero edits in 9/12 batches. Its accepted raw snapshot still caused 0/24 release failures. The exact-support result therefore serves as a batch audit. The physical experiment evaluates the fixed target and bounded attacker without establishing joint gate optimality of the tanh rule.

Table 7: Commit availability and offline computation on the independent 144-state candidate pool. Common admission requires both freeze and the committed snapshot to pass the 100-step, 0.003-guard certificate.

Seed	Common/144	Commit frac.	Offline (s)
2040	143/144 (99.3%)	0.55	8.49
2041	144/144 (100.0%)	0.40	9.32
2042	130/144 (90.3%)	0.70	8.50

E Additional Boundaries and Validity Checks

This appendix expands the boundary audits summarized in Section 6.

Trusted-state-anchor infrastructure. A trusted state anchor shrinks certificate uncertainty but does not sanitize the reward record. Varying the anchor period $P \in \{1, 3, 6, 12\}$ (12, 4, 2, or 1 calls over 12 updates), every condition retains all 16 certificate centers, has 0/24 violations on new paired blocks, and improves no reward over freeze; retained fractions stay at 0.55/0.40/0.70 for $P \leq 6$ and become 0.50/0.40/0.65 at $P = 12$. Hardware latency, energy, authentication, and monetary cost are not measured.

Official PPO. The official pretrained PPO policy is safe on 72/72 held-out rollouts, while a separately constructed malicious target is unsafe on 72/72. Under the locked $L_\infty \leq 0.5$ reward budget, the 16-batch checkpoint has 0/24 clean and 1/24 poisoned failures; at 24 batches both have 0/24. The pre-specified physical and two-checkpoint margin conditions failed, so the planned multi-run extension was not executed. The audit showed checkpoint-level update redirection without a stable deep-PPO unsafe-policy effect.

Benign adaptation utility. In the locked benign-utility audit under a 0.003 N differential actuator bias, freeze, clean adaptation, and LifecycleGate all have 0/12 violations and mean rewards -0.03617 , -0.04714 , and -0.08097 . LifecycleGate accepts a nonzero update in 11/12 batches (mean fraction 0.8208) but improves no paired rollout over freeze. This met the preregistered stopping criterion, so the planned three-run extension was not executed. Parameter motion and certificate acceptance did not establish useful adaptation in this setting.

Measurement. The CasADi gates use fixed finite state sets; physical outcomes use independent PyBullet rollouts. They are sampled finite-horizon certificates, not continuous-state invariant-set proofs. For the benign shift, maximum action implementation error is 2.61×10^{-11} N. A corrected calibration metric left run indices, states, bias grid, and threshold unchanged. Realized state violations and pre-clip saturation are reported separately.

F Additional Third-Party Batteries

This appendix reports the learner-family, attack-budget, and second-environment batteries summarized in Section 5.

F.1 Learner Boundary

The same contrast attack ($\delta=2$) was evaluated on the third-party PPO pipeline with action masking and sudden retirement. It raised post-removal violations in one of three paired runs, from 1178 to 2439 (McNemar exact $p = 2.6 \times 10^{-149}$). The first-violation episode changed from 6 to 0. The other two runs were unchanged ($p = 1.00$ and 0.96). In a separate ten-run shield-on-only isolation battery, the run indexed by configured seed 101 was positive ($0.219 \rightarrow 0.754$ at retirement; McNemar exact $p = 3.7 \times 10^{-113}$), while the other nine entered a high-violation ceiling regime (clean 0.728–0.760) and were either protective or unchanged. Figure 5 reports the full battery. A ten-run full-phase PPO battery likewise placed every clean policy near the retirement ceiling (0.73 – 0.77) and yielded no positive outcome on the primary retirement metric. Its final metric captured a different effect: all ten poisoned policies remained unsafe (0.49 – 0.77), whereas nine clean policies recovered to approximately 0.22 under continued learning.

A ten-run shield-on-only battery on off-policy soft actor-critic (SAC) provided a complementary boundary. Three runs were positive ($0.207 \rightarrow 0.561$, $0.218 \rightarrow 0.513$, and $0.479 \rightarrow 0.954$; paired McNemar exact $p < 10^{-40}$ for each), three were protective, and four were unchanged. All ten clean SAC policies had low retirement disagreement rather than occupying the ceiling regime. Four extension runs generated on a second host were non-positive. The fixed attack therefore affected individual PPO and SAC training realizations without producing reliable success within either family.

F.2 Attack Budget and State Dependence

On the full-phase REINFORCE protocol, the effect varied with attack shape, budget, and training realization. In the realization indexed by configured seed 2, $\delta=2$ raised post-removal violations from 1127 to 3762 ($3.3\times$; McNemar exact p below numerical reporting precision). Increasing the budget to $\delta=10$ produced 3766 violations. In the realization indexed by seed 3, $\delta=2$ was unchanged (2482 vs. 2471; $p = 0.84$), whereas $\delta=10$ raised violations to 3700 ($p = 8.4 \times 10^{-138}$). The realization indexed by seed 1 began near the unsafe ceiling (75.7%) and showed no added damage at either budget. Shape controls were also state-dependent. The bias variant was protective in the ceiling realization. The risk variant was protective in two realizations and raised violations from 1127 to 2457 in the realization indexed by seed 2.

An independent Linux-host batch with identical dependency versions contained one low-clean-violation realization

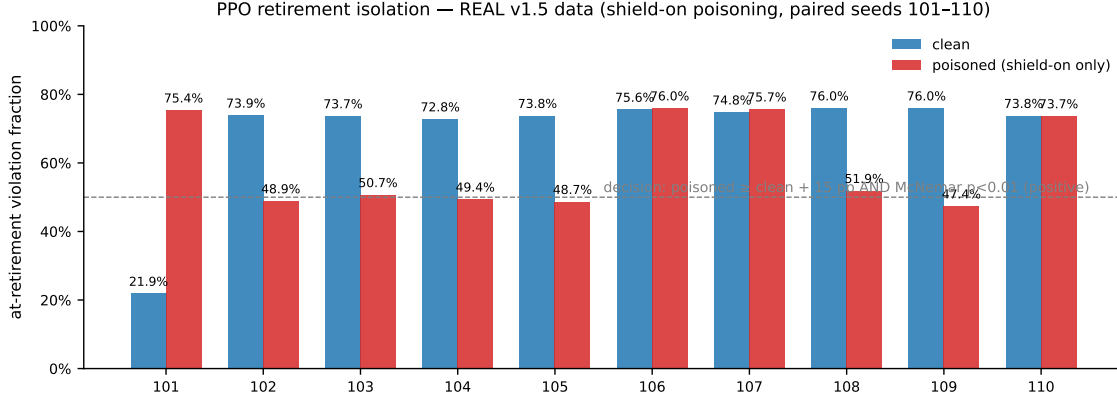


Figure 5: PPO retirement-boundary isolation on the third-party lifecycle (shield-on-only contrast poisoning, $\delta=2$, paired runs indexed by run indices 101–110, sudden retirement). At-retirement violation fraction of the raw policy for clean and poisoned runs. The run indexed by 101 is positive ($0.219 \rightarrow 0.754$; McNemar exact $p = 3.7 \times 10^{-113}$); the other nine runs land in the unsafe ceiling regime (clean at-retirement $0.728\text{--}0.760$) with no paired headroom, six protective by the paired test and three unchanged. The dashed line marks the locked decision boundary (poisoned at least 0.15 above clean with paired McNemar $p < 0.01$ positive).

and two ceiling realizations. The former showed the contrast effect at retirement ($0.232 \rightarrow 0.501$) and after removal ($1157 \rightarrow 2451$; $p = 4.6 \times 10^{-159}$). The latter two were unchanged at $\delta=2$ and protective at $\delta=10$. The run index of the low-clean-violation regime differed across hosts. Repeating the clean configuration also entered a different trajectory mode (0.776 rather than 0.232 at retirement). Dose comparisons are therefore confined to the locked local batch. The independent batch shows that attackability follows the realized retirement state more closely than the nominal seed identifier.

F.3 Environment Boundary

On a second gridworld of the same third-party suite (surveillance, $N=6$, $R=3$), a clean reproduction again satisfied the authors’ lifecycle ordering: shield retained $0/0$, smooth 0.129 , sudden 0.246 , and no-shield 0.252 (retained $<$ smooth $\ll$ sudden $\leq$ no-shield). The fixed contrast attack at $\delta=2$ raised post-removal violations in one of three independent paired runs ($0.000 \rightarrow 0.272$, McNemar exact $p = 2.6 \times 10^{-82}$) and was protective in the other two ($0.267 \rightarrow 0.000$, $p = 8.4 \times 10^{-81}$; $0.135 \rightarrow 0.038$, $p = 1.3 \times 10^{-15}$). The lifecycle ordering extended to the second environment, whereas the exploit effect remained environment- and realization-dependent.

F.4 Opportunity-Region Cross-Tabulation

Figure 7 plots the coarse two-by-two cross-tabulation summarized in Section 5: across 53 locked paired runs spanning both scopes, all three learner families, and both domains, 12 of 20 runs with clean at-retirement violation below 0.5 were positive, whereas none of 33 ceiling runs was positive.

F.5 Family-Stratified Counts and Band Chronology

Family-stratified counts clarify the AUC reported in Section 5. REINFORCE had seven positive low-band runs and 13 non-positive high-band runs. PPO had one positive low-band run and 19 non-positive high-band runs. All ten SAC runs were low-band, with three positive and seven non-positive outcomes.

The protocol chronology bounds the statistical interpretation. The $0.06/0.09$ bands were fixed after analysis of 46 existing obstacle rows and before four SAC extension pairs with run indices 407–410. All four extensions were low-band and non-positive. They strengthened the non-sufficiency result without prospectively testing high-disagreement exclusion. The marker is also obstacle-scoped: two protective runs in the second environment had clean disagreement 0.476 and 0.157 despite clean violation below 0.5 .

G LifecycleGate Semantic Benchmarks

This appendix contains the low-dimensional semantic benchmarks that distinguish current-action filtering from update admission. They instantiate the certificate semantics of Section 4 rather than the authority-transition attack evaluated in Section 6.

G.1 Setup

We instantiated the linear system of Appendix B with $\lambda = 1.2$, $u_{\max} = 0.2$, and $S = [-1, 1]$. FDI induced the ambiguity set $X_{\mathcal{A}}(h) = [-\rho, \rho]$. The common certified kernel for invariant

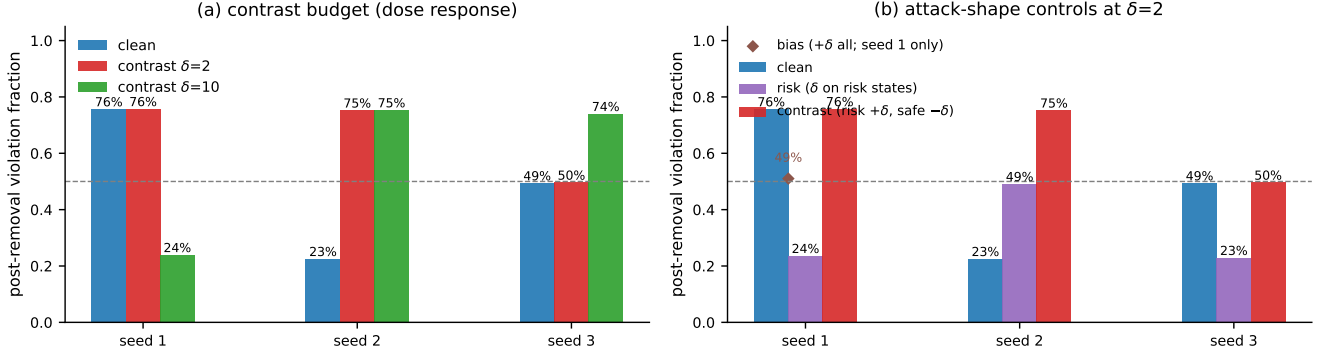


Figure 6: Budget- and state-dependent effects on the full-phase REINFORCE protocol (paired within one locked code version). (a) Contrast attack dose response; (b) attack-shape negative controls at $\delta=2$. The run-index-2 realization saturates at $\delta=2$; the run-index-3 realization requires $\delta=10$; the ceiling realization shows no additional damage at either budget, and bias/risk controls are protective or strictly weaker.

Table 8: Certified kernel for the one-dimensional ambiguity set.

ρ	$\lambda\rho$	$K(\rho)$	Gate
0.600	0.720	$[-0.200, 0.200]$	allow
0.800	0.960	$[-0.040, 0.040]$	constrain
0.833	1.000	$\{0\}$	boundary
0.900	1.080	$\emptyset$	freeze/reject
1.000	1.200	$\emptyset$	freeze/reject

certificate $I = S$ is

$$K(\rho) = [-u_{\max}, u_{\max}] \cap [-1 + \lambda\rho, 1 - \lambda\rho]. \quad (28)$$

The kernel is nonempty exactly when $\lambda\rho \leq 1$.

G.2 Learning-Freeze Frontier

The frontier occurs at $\rho = 1/\lambda = 0.833$. Table 8 shows the exact kernel values for representative ambiguity radii.

At $\rho = 0.9$, states $+0.9$ and -0.9 are individually controllable when their sign is trusted. Their safe intervals are $[-0.2, -0.08]$ and $[0.08, 0.2]$, respectively. The empty intersection prevents a common history-dependent action from being certified.

G.3 Trusted Anchors and Learning-Aware FDI

A trusted state anchor can restore learning by shrinking the plausible-state set. At $\rho = 0.9$, a trusted sign anchor reduces the set to $[0, 0.9]$ or $[-0.9, 0]$. The corresponding kernels, $[-0.2, -0.08]$ and $[0.08, 0.2]$, are nonempty.

Learning-aware FDI creates the complementary failure mode. For a candidate action $u = 0.1$, the gate allows the update at $\rho = 0.6$, projects it to 0.04 at $\rho = 0.8$, and rejects it

at $\rho = 0.9$. The attacked history therefore determines whether the same candidate is allowed, constrained, or rejected.

G.4 Lifecycle Benchmark Results

We compared five mechanisms over five update steps: an ungated learner, a nominal action filter, a robust action filter, always-freeze, and LifecycleGate with projection. The one-dimensional benchmark used the system above. The linear tank benchmark used the level-deviation model

$$x_{t+1} = Ax_t + Bu_t, \quad A = \begin{bmatrix} 1.08 & 0.04 \\ 0.02 & 1.03 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0.1 \end{bmatrix}, \quad (29)$$

with $|u_t| \leq 0.15$ and $x_t \in [-1, 1]^2$. The nonlinear tank benchmark uses square-root inter-tank and drain flows,

$$\begin{aligned} h_{1,t+1} &= h_{1,t} + \Delta t(q_0 + u_t - q_{12}(h_t) - d_1(h_{1,t})), \\ h_{2,t+1} &= h_{2,t} + \Delta t(q_{12}(h_t) - d_2(h_{2,t})), \end{aligned} \quad (30)$$

where q_{12} and d_i are square-root flow terms. Its interval observer returned a box around the nominal attacked history. The one-step kernel was computed over a grid within that box. In all benchmarks, learning-aware FDI pushed the learner toward a larger positive action parameter.

Figure 8 separates current-action filtering from update certification. At $\rho = 0.8$, the nominal filter had unsafe certified rate 1.0. The robust action filter had stale-policy rate 1.0 because the learner parameter left $K(\rho)$ after action projection. LifecycleGate had zero unsafe or stale operation and retained one of five updates. At $\rho = 0.9$, the kernel was empty and LifecycleGate froze learning.

The linear tank benchmark reproduced the same pattern. At $\rho = 0.8$, nominal filtering had unsafe certified rate 0.8, and robust action filtering had stale-policy rate 0.8. LifecycleGate kept both rates at zero and allowed two of five updates. It allowed one constrained update at $\rho = 0.85$ and froze at the empty-kernel point $\rho = 0.92$.

Susceptibility rule: positive only when clean at-ret < 0.5

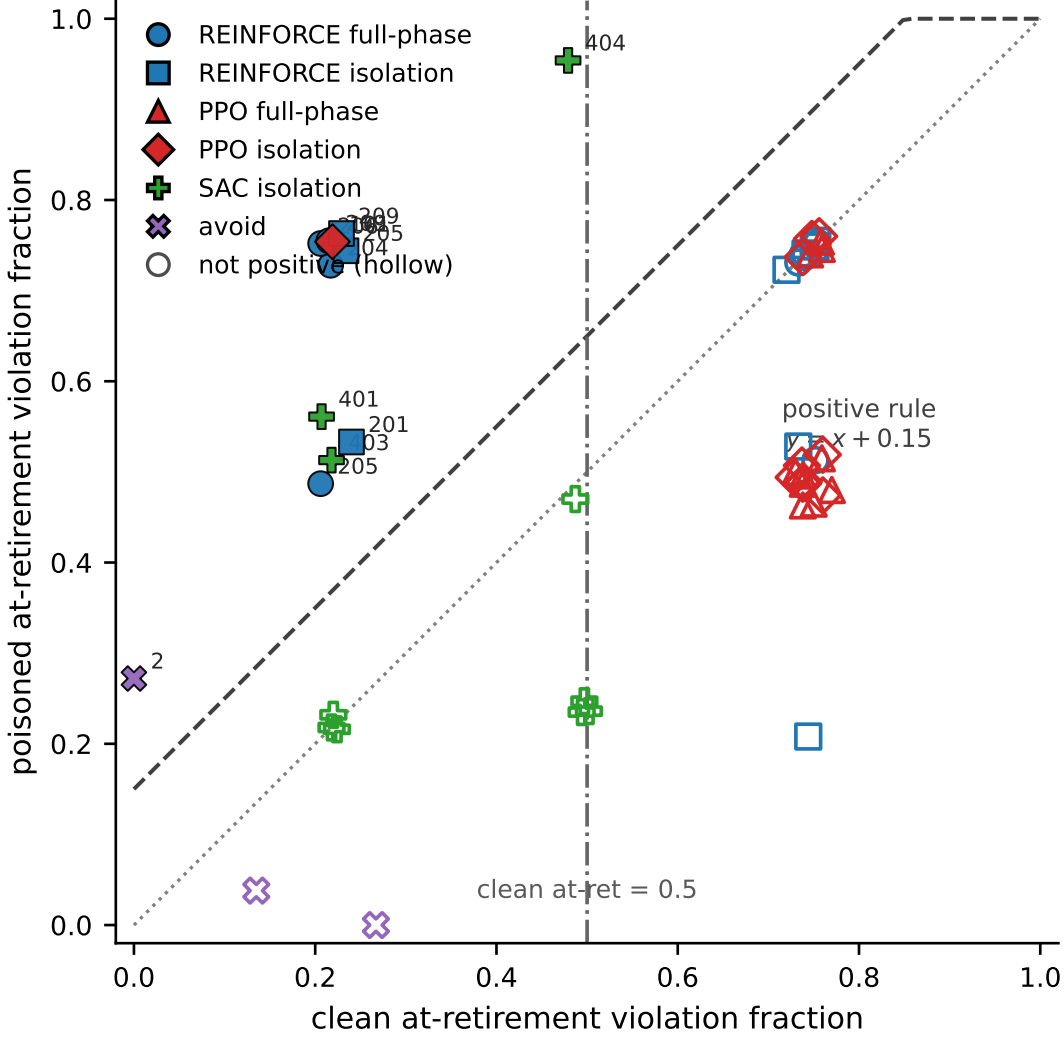


Figure 7: Two-by-two opportunity-region cross-tabulation across all locked paired runs (both scopes, all three learner families, both domains). Each point is one paired clean/poisoned run; filled markers meet the positive rule (poisoned at-retirement at least 0.15 above clean at retirement with paired McNemar exact $p < 0.01$), hollow markers do not. The dashed rule line is $y = x + 0.15$ and the vertical dashed line marks the clean at-retirement 0.5 boundary: positive outcomes concentrate below 0.5, and no ceiling run is positive.

The nonlinear tank benchmark preserved the separation with kernels computed from sampled nonlinear dynamics. At $\rho = 0.2$, nominal filtering was unsafe in four of five steps, and robust filtering was stale in four. LifecycleGate kept both rates at zero while allowing two constrained updates. At $\rho = 0.45$, the interval-observer box admitted no common safe action and LifecycleGate froze the update.

G.5 Parameter-Level Update Gate

We also instantiated the gate at the parameter level for linear policy $u = kz + b$. Each future attacked observation z induced a safe-action kernel over ambiguity interval $[z - \rho, z + \rho]$. Requiring $kz + b$ to lie in every kernel produced linear halfspace constraints on (k, b) . The parameter gate projected poisoned candidates onto this certified set.

Figure 9 shows the parameter-level frontier. At $\rho = 0.7$, current-action filtering had stale-policy rate 1.0. The param-

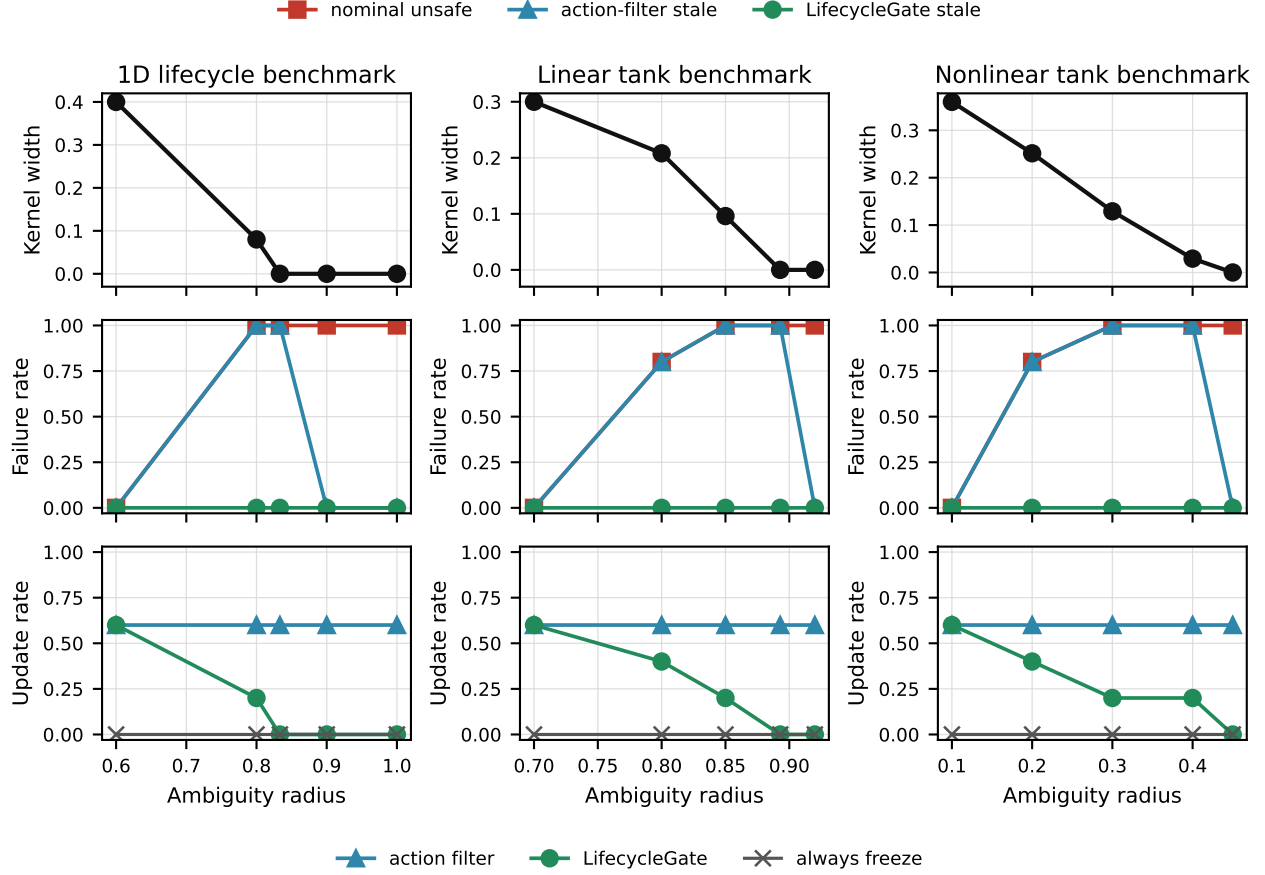


Figure 8: Lifecycle-gate benchmark results. Increasing ambiguity shrinks the certified kernels of the one-dimensional, linear-tank, and nonlinear-tank models. LifecycleGate constrains or freezes updates when action-only filters would leave the updated policy outside the kernel.

ter gate kept the rate at zero and permitted two of five updates. It permitted one constrained update at $\rho = 0.8$ and froze at $\rho = 0.85$.

We applied the same construction to policy $u = k_1 z_1 + k_2 z_2 + b$. Each grid observation induced a plausible-state box and a one-step safe-action interval. These intervals produced halfspaces in (k_1, k_2, b) . At $\rho = 0.5$, current-action filtering had stale-policy rate 1.0. The parameter gate kept the rate at zero and admitted all five projected updates.

G.6 Adaptive Attacker Stress Test

We next allow a finite-search white-box attacker to select among poisoned update directions. Its objectives maximize stale-policy, unsafe-action, or forced-freeze outcomes. For forced freeze, the attacker also selects the budget-feasible ambiguity radius that most reduces the certified set.

Figure 11 preserves the lifecycle separation under adaptive direction search. At high ambiguity, the robust action filter protected the applied action but left the raw policy stale. LifecycleGate kept unsafe, stale, and uncertified-learning rates at zero while allowing feasible projected updates. At $\rho = 0.84$,

the forced-freeze objective made the common kernel empty. The robust filter recorded five uncertified learner updates, whereas LifecycleGate performed none.

G.7 Safe-Control-Gym Cartpole Semantics

We next instantiate the lifecycle condition in the Safe-Control-Gym cartpole benchmark. The official linear-quadratic regulator (LQR) is augmented with a two-parameter linear residual learner. The attacker injects continuous observation FDI and optimizes a cross-entropy-method (CEM) objective over stale-policy, unsafe-action, and forced-freeze margins. The attacked observation z and radius ρ induce the plausible box $X_{\mathcal{A}}(z) = \{x : \|x - z\|_{\infty} \leq \rho\}$. We sample this box and evaluate the official CasADi cartpole dynamics over an action grid to compute $K_{\Gamma}(z)$. A reference run uses the official cartpole CBF and its sampled Lie-derivative condition.

This experiment is a direct semantic implementation of Theorem 1 and Proposition 2. The future-history abstraction $\mathcal{Z}$ is a local grid of future attacked observations around the current attacked observation. For each $z' \in \mathcal{Z}$, the robust kernel produces an interval $K_{\Gamma}(z') = [\ell_{z'}, u_{z'}]$. The LQR term is fixed,

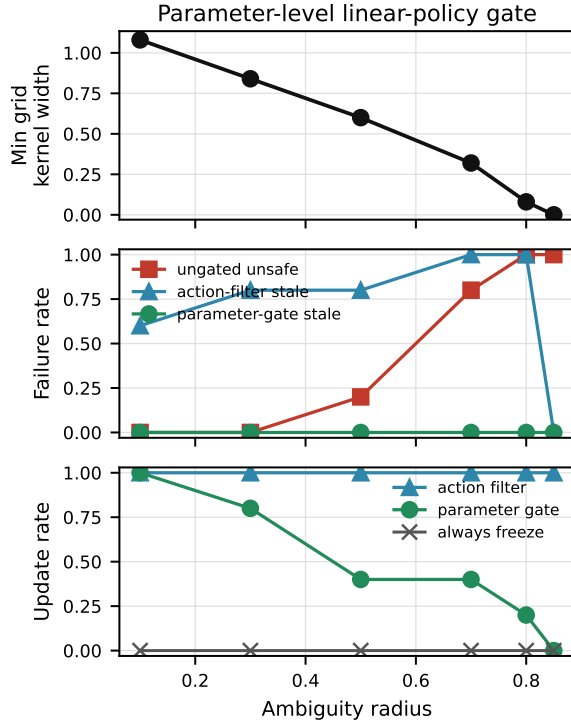


Figure 9: Parameter-level gate for linear policy $u = kz + b$. Current-action filtering leaves the future action map stale, whereas projection onto the certified halfspace set constrains the full update.

while the residual learner is linear in $(\theta_{\text{gain}}, \theta_{\text{bias}})$, so requiring the updated residual policy to stay inside all future kernels yields the halfspace constraints used by LifecycleGate.

Figure 12 separates three effects. Action filtering removed current constraint violations without certifying the updated residual policy. Always-freeze avoided stale certification but admitted no learning. LifecycleGate projected feasible poisoned updates into the future-history halfspace set, preserving update availability before the freeze frontier. At larger ambiguity radii, the kernel became empty and LifecycleGate froze the update. These results instantiate the certificate semantics in a standard control benchmark; the evaluation does not represent an industrial deployment.

H Certificate Coverage and Model Mismatch

Reporting zero violations only on certificate-admitted states can be vacuous. We therefore lock a separate CasADi–PyBullet confusion audit before execution. It reconstructs the 12 clean, poisoned, commit, and freeze snapshots from the original traces, draws 144 states from three disjoint source seeds, performs no certificate admission, and compares horizons 20/50/100 and guard margins 0/0.001/0.003/0.005/0.01. The primary configuration is the pre-existing 100-step, 0.003-

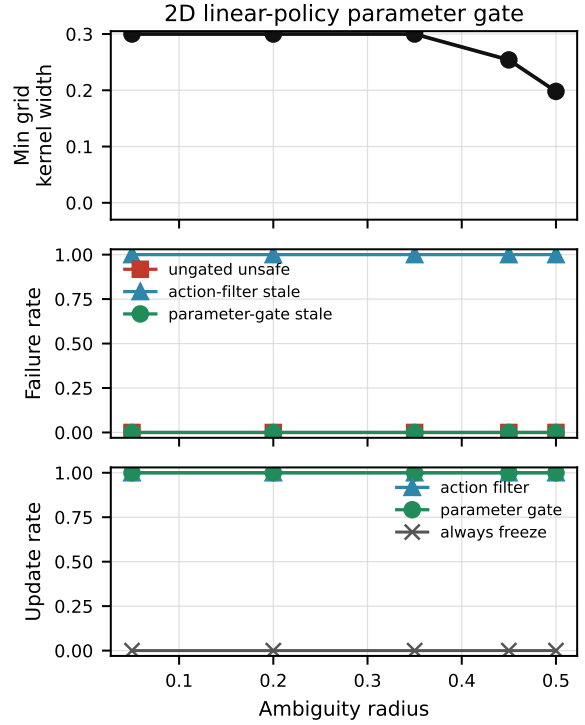


Figure 10: Parameter-level gate for the two-dimensional linear tank policy. The certified set intersects future-history halfspaces, and the gate projects the poisoned update into that set.

guard commit configuration; the grid is characterization rather than a selector.

Table 9 evaluates 1728 snapshot–state pairs. CasADi certifies 1293; none violates the PyBullet state box within 100 steps. Clean, commit, and freeze coverage is 0.986, 0.944, and 1.000. The raw poisoned snapshots physically violate 403/432 pairs and have certificate coverage 0.062, so their low coverage is informative rejection rather than vacuous evidence for a safe deployed snapshot. Across all mechanisms the certificate conservatively rejects five physically safe pairs. Maximum action-interface error and actuator saturation are zero.

The grid exposes a model-mismatch boundary. Without a guard, poisoned snapshots produce 1, 2, and 1 false acceptances at horizons 20, 50, and 100. Every locked positive guard removes observed false acceptance, with increasing false rejection as the guard grows. The value 0.003 was the declared primary configuration before this audit. Zero observed false acceptance on the finite evaluation distribution does not establish continuous-state invariance.

I Unified Paired Security–Availability Frontier

Table 9: CasADi-PyBullet coverage at the pre-existing 100-step, 0.003-guard configuration. Each row pools three learner snapshots over the same 144 states.

Snapshot	Pairs	Physical viol.	Certified	False accept	False reject	Coverage
Clean REINFORCE	432	6	426	0	0	0.986
Poisoned action-only	432	403	27	0	2	0.062
Poisoned commit	432	21	408	0	3	0.944
Always-freeze	432	0	432	0	0	1.000

Table 10: Unified paired security–availability frontier. Wilson 95% intervals are $[0, 0.138]$ for every 0/24 violation count and $[0.798, 0.993]$ for poisoned raw. Intervention is a fraction of executed steps. † denotes reward truncated at the first safety failure and therefore not a utility comparison.

Mechanism	Viol./rollouts	Complete	Mean reward	Interv.	Latency med./p95 (ms)
Clean REINFORCE	0/24	24/24	−0.024	0.0%	0.027/0.029
Poisoned raw	23/24	1/24	−0.189†	0.0%	0.027/0.029
Poisoned commit	0/24	24/24	−0.056	0.0%	0.026/0.029
Always-freeze	0/24	24/24	−0.023	0.0%	0.026/0.028
Permanent shield	0/24	24/24	−0.195	21.3%	221.925/238.216
Official MPSC	0/24	24/24	−0.457	100.0%	59.657/76.055

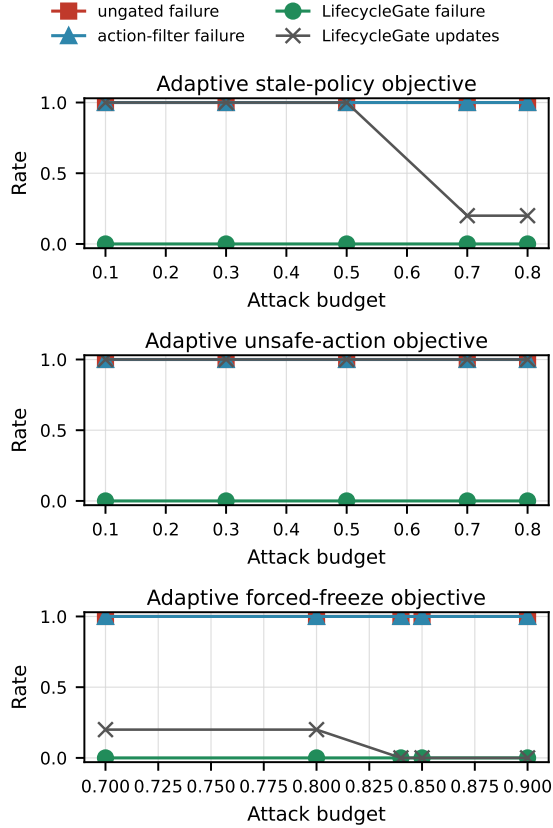


Figure 11: Finite-search attacker against the scalar parameter gate. The attacker selects update directions or ambiguity radii for three objectives. LifecycleGate freezes when the certified kernel is empty.

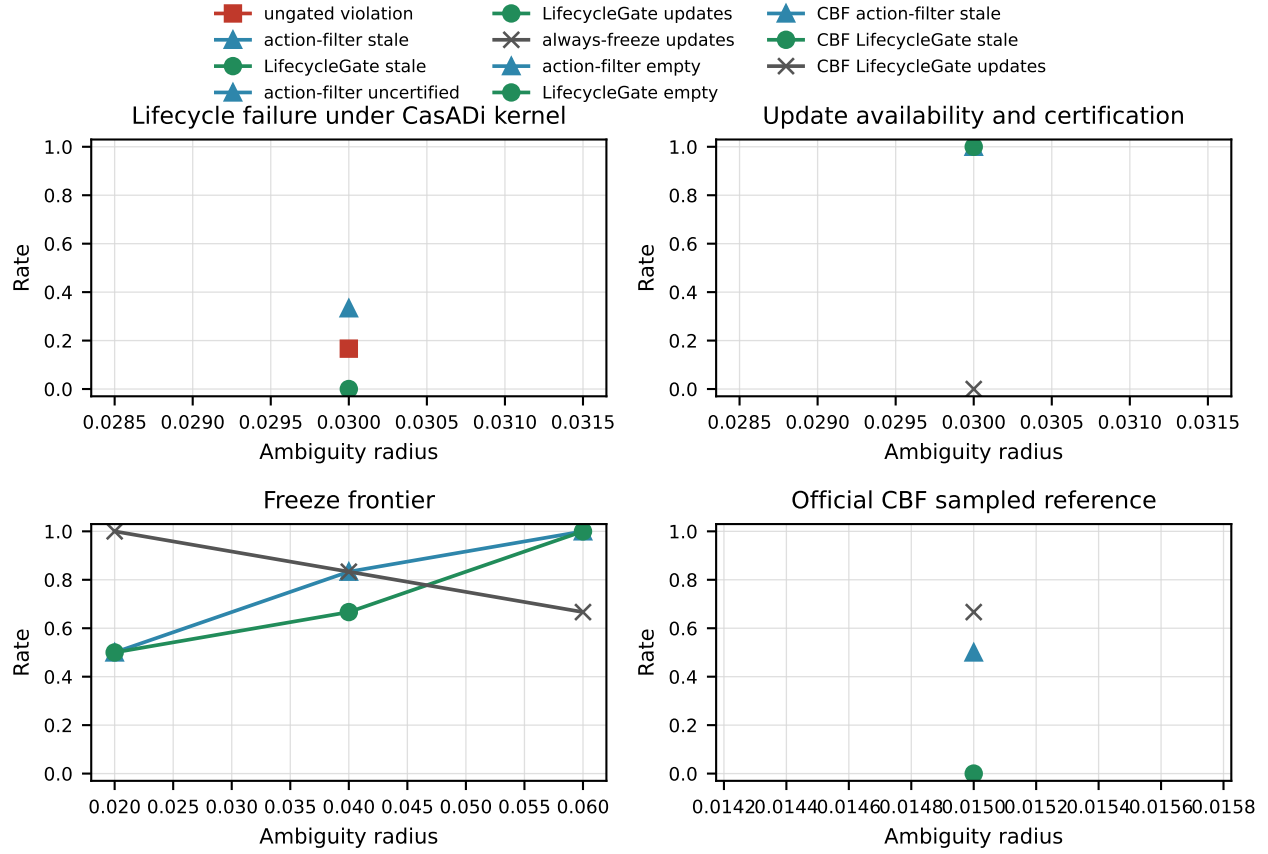


Figure 12: Safe-Control-Gym cartpole lifecycle evidence from a two-run sweep. The panels compare the CasADi robust kernel, an Euler attack-search surrogate, and a sampled CBF reference. At $\rho = 0.03$, LifecycleGate eliminated stale and uncertified updates while retaining update availability. The kernel became empty as ambiguity increased.